TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO BÀI TẬP LỚN

ĐỀ TÀI:

HỆ THỐNG QUẢN LÝ DỰ ÁN NHÓM CÓ TÍCH HỢP AI

Giảng viên hướng dẫn: Hà Thị Thanh
Sinh viên thực hiện: Phạm Văn Huynh
Đàm Đức Đôn
Phạm Minh Tú
Lớp: Kỹ thuật máy tính K23A

Thái Nguyên, năm 2026

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO BÀI TẬP LỚN

ĐỀ TÀI:

HỆ THỐNG QUẢN LÝ DỰ ÁN NHÓM CÓ TÍCH HỢP AI

Giảng viên hướng dẫn: Hà Thị Thanh
Sinh viên thực hiện: Phạm Văn Huynh
Đàm Đức Đôn
Phạm Minh Tú
Lớp: Kỹ thuật máy tính K23A

Thái Nguyên, năm 2026

LỜI CẢM ƠN

Nhóm chúng em xin gửi lời cảm ơn chân thành tới các thầy cô Khoa Công nghệ Thông tin, Trường Đại học Công nghệ Thông tin và Truyền thông, đã truyền đạt kiến thức và hướng dẫn chúng em trong suốt học phần. Đặc biệt, chúng em cảm ơn giảng viên hướng dẫn đã góp ý định hướng cho dự án—trong đó có gợi ý hỗ trợ quy trình Scrum, sau này trở thành một trong những chức năng chính của hệ thống.

Mặc dù đã cố gắng, dự án và báo cáo chắc chắn vẫn còn thiếu sót. Chúng em rất mong nhận được ý kiến đóng góp của thầy cô để hoàn thiện hơn.

Thái Nguyên, năm 2026

Nhóm thực hiện

Phạm Văn Huynh Đàm Đức Đôn Phạm Minh Tú

MỤC LỤC

LỜI CẢM ƠN **i**

DANH SÁCH BẢNG **iv**

DANH SÁCH HÌNH VẼ **v**

DANH MỤC TỪ VIẾT TẮT **vi**

MỞ ĐẦU **1**
0.1. Lý do chọn đề tài . . . 1
0.2. Mục tiêu . . . 1
0.3. Phạm vi . . . 1
0.4. Phương pháp thực hiện . . . 2
0.5. Tổ chức nhóm và phân công nhiệm vụ . . . 2
0.6. Cấu trúc báo cáo . . . 3

CHƯƠNG 1: CƠ SỞ LÝ THUYẾT **4**
1.1. Quản lý dự án cho nhóm phần mềm nhỏ . . . 4
1.1.1. Kanban . . . 4
1.1.2. Scrum . . . 4
1.2. Đồ thị phụ thuộc và đường găng . . . 4
1.2.1. Sắp xếp topo . . . 4
1.2.2. Đường găng . . . 5
1.3. Phần mềm local-first . . . 5
1.4. Mô hình ngôn ngữ lớn và đầu ra có cấu trúc . . . 5
1.5. Công nghệ sử dụng . . . 6
1.6. Tài liệu như một hệ thống kiểm soát . . . 6

CHƯƠNG 2: KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU **7**
2.1. Khảo sát các công cụ hiện có . . . 7
2.2. Tác nhân . . . 8
2.3. Phân tích nghiệp vụ . . . 9
2.3.1. Quy trình hiện tại và các vấn đề . . . 9
2.3.2. Quy trình mục tiêu . . . 9
2.3.3. Vòng đời của task . . . 9
2.3.4. Quy tắc nghiệp vụ . . . 9
2.4. Yêu cầu chức năng . . . 10
2.4.1. Quy trình Scrum . . . 11

2.5. Yêu cầu phi chức năng 12

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG 13
3.1. Kiến trúc tổng thể 13
3.2. Mô hình dữ liệu 14
3.3. Thiết kế API 14
3.4. Thiết kế phân quyền 15
3.5. Chuỗi dự phòng AI 15
3.6. Chính sách ngoại tuyến và đồng bộ 16
3.7. Thiết kế giao diện 16

CHƯƠNG 4: TRIỂN KHAI VÀ KIỂM THỬ 17
4.1. Môi trường phát triển 17
4.2. Các chức năng đã cài đặt 17
4.2.1. Trang giới thiệu và xác thực 17
4.2.2. Dự án, thành viên và vai trò 18
4.2.3. Bảng Kanban và dạng bảng 19
4.2.4. Chi tiết task: thảo luận, minh chứng và lịch sử 20
4.2.5. Scrum 21
4.2.6. Các công cụ AI 24
4.2.7. Đồ thị phụ thuộc, thông báo và hồ sơ 26
4.3. Kiểm thử 27
4.3.1. Chiến lược 27
4.3.2. Kết quả 28
4.3.3. Các lỗi đã phát hiện 28
4.4. Triển khai 29
5.1. Từ trợ lý đến nhà máy 30
5.2. Kỹ thuật ngữ cảnh 30
5.3. Nhà máy: các trạm và trách nhiệm 31
5.4. Rào chắn và bảo mật 32
5.5. Kiểm thử đơn vị và kiểm thử tích hợp 34
5.6. Đóng gói, vận hành và theo dõi sản phẩm 34
5.6.1. Đóng gói 34
5.6.2. Vận hành 34
5.6.3. Theo dõi 34
5.7. Đánh giá 35

KẾT LUẬN 36

TÀI LIỆU THAM KHẢO 38

NHẬN XÉT CỦA GIẢNG VIÊN 39

DANH SÁCH BẢNG

Bảng 1. Phân công nhiệm vụ các thành viên 2
Bảng 1.1. Các công nghệ sử dụng 6
Bảng 2.1. So sánh với các công cụ phổ biến 7
Bảng 2.2. Các tác nhân của hệ thống 8
Bảng 2.3. Các quy tắc nghiệp vụ chính 10
Bảng 2.4. Các nhóm yêu cầu chức năng 10
Bảng 2.5. Các yêu cầu phi chức năng chính 12
Bảng 3.1. Tổng quan API 15
Bảng 4.1. Môi trường phát triển và quy mô dự án 17
Bảng 4.2. Kết quả kiểm thử 28
Bảng 4.3. Một số lỗi đã phát hiện và sửa 28
Bảng 5.1. Các tài liệu agent đọc và câu hỏi mỗi tài liệu trả lời 31
Bảng 5.2. Các trạm của nhà máy 31
Bảng 5.3. Một số quy tắc rào chắn 33
Bảng 5.4. Các mức kiểm thử 34

DANH SÁCH HÌNH VẼ

Hình 2.1. Các ca sử dụng chính (PM làm được mọi việc của Thành viên) . . 8
Hình 2.2. Quy trình nghiệp vụ mục tiêu của một nhóm dự án 9
Hình 2.3. Vòng trạng thái task (phím Space chuyển một bước) 9
Hình 2.4. Vòng đời sprint 11
Hình 3.1. Kiến trúc triển khai và các thành phần 13
Hình 3.2. Các thực thể (10 bảng) 14
Hình 3.3. Chuỗi dự phòng AI ba tầng 16
Hình 4.1. Trang giới thiệu 18
Hình 4.2. Hộp thoại đăng nhập 18
Hình 4.3. Bảng điều khiển dự án với vai trò theo từng dự án 19
Hình 4.4. Bảng Kanban 19
Hình 4.5. Dạng bảng 20
Hình 4.6. Phím tắt 20
Hình 4.7. Chi tiết task với thảo luận và lịch sử 21
Hình 4.8. Bảng Scrum: vòng đời sprint 22
Hình 4.9. Bảng Scrum: lập kế hoạch backlog 22
Hình 4.10. Bảng Scrum: biểu đồ burndown 23
Hình 4.11. Bảng Scrum: velocity 23
Hình 4.12. Bảng Scrum: retrospective do AI viết 24
Hình 4.13. AI tóm tắt tiến độ tuần 24
Hình 4.14. AI tạo biên bản họp 25
Hình 4.15. AI phân tích khối lượng và gợi ý giao việc 25
Hình 4.16. AI chia nhỏ mục tiêu 26
Hình 4.17. Thứ tự thực hiện DAG và đường găng 26
Hình 4.18. Luồng thông báo 27
Hình 4.19. Trang hồ sơ 27
Hình 5.1. Nhà máy agent AI: ngữ cảnh đầu vào, các trạm nối tiếp, rào chắn quanh mọi bước 30
Hình 5.2. Mọi lần sửa file và mọi lệnh đều qua chính sách trước khi thực hiện 32

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	**Diễn giải**
AI	Trí tuệ nhân tạo (Artificial Intelligence)
API	Application Programming Interface
BR	Quy tắc nghiệp vụ (Business Rule)
CI	Tích hợp liên tục (Continuous Integration)
CORS	Cross-Origin Resource Sharing
CPM	Phương pháp đường găng (Critical Path Method)
CRUD	Create, Read, Update, Delete
DAG	Đồ thị có hướng không chu trình (Directed Acyclic Graph)
DEC	Bản ghi quyết định (trong sổ tay phát triển D7)
E2E	Kiểm thử đầu-cuối (End-to-end)
i18n	Quốc tế hoá (đa ngôn ngữ)
JWT	JSON Web Token
LLM	Mô hình ngôn ngữ lớn (Large Language Model)
ORM	Object-Relational Mapping
PM	Quản lý dự án (vai trò theo từng dự án)
REST	Representational State Transfer
RTM	Ma trận truy vết yêu cầu
SDLC	Vòng đời phát triển phần mềm (Software Development Life Cycle)
SPA	Ứng dụng web một trang
TLS	Transport Layer Security
UI / UX	Giao diện / Trải nghiệm người dùng
WIP	Công việc đang thực hiện

MỞ ĐẦU

0.1. Lý do chọn đề tài

Các nhóm phát triển phần mềm nhỏ—đặc biệt là nhóm sinh viên—thường quản lý công việc bằng tin nhắn, bảng tính hoặc một công cụ đa năng như Trello hay Jira. Mỗi lựa chọn đều có cái giá của nó. Tin nhắn làm thất lạc thông tin; bảng tính không biết gì về trạng thái, phụ thuộc hay hạn chót; còn các công cụ lớn thì nặng, phụ thuộc chuột, luôn cần kết nối mạng và thu phí đúng những chức năng nhóm cần như báo cáo Scrum. Trong khi đó, hai xu hướng giúp một công cụ tốt hơn trở nên khả thi: trình duyệt giờ có thể lưu toàn bộ bản sao làm việc của dự án ngay trên máy (ý tưởng *local-first* [1]), và các mô hình ngôn ngữ lớn có thể biến văn bản tự do—biên bản họp, một mục tiêu còn mơ hồ—thành dữ liệu có cấu trúc, có thể hành động được.

Đề tài xây dựng một hệ thống quản lý dự án nhóm quanh ba ý tưởng: mọi thao tác đều làm được bằng bàn phím; bảng công việc vẫn hoạt động khi mất mạng; và AI đảm nhận các phần tẻ nhạt của quản lý (tóm tắt, biên bản, giao việc, chia nhỏ công việc, retrospective cuối sprint) nhưng luôn trả về kết quả có cấu trúc, kiểm tra được.

0.2. Mục tiêu

- Xây dựng ứng dụng web cho phép người dùng đăng ký, tạo dự án và mời thành viên, với vai trò (Quản lý dự án hoặc Thành viên) được gán *theo từng dự án* chứ không theo tài khoản.
- Cung cấp bảng Kanban 4 cột và chế độ xem dạng bảng, điều khiển hoàn toàn bằng bàn phím, có phụ thuộc giữa các task và đường găng được tính tự động.
- Hỗ trợ quy trình Scrum: lập kế hoạch sprint, product backlog, bắt đầu và kết thúc sprint, biểu đồ burndown và velocity, retrospective do AI viết.
- Tích hợp mô hình ngôn ngữ lớn (Groq, model `openai/gpt-oss-120b`) cho năm chức năng AI, với cơ chế dự phòng ba tầng để ứng dụng không bao giờ lỗi khi dịch vụ AI không sẵn sàng.
- Giữ dự án cá nhân dùng được đầy đủ khi ngoại tuyến, đồng thời ngăn chỉnh sửa xung đột khi ngoại tuyến ở dự án có nhiều thành viên.
- Triển khai bản chạy cấu hình production trên tên miền công khai để nhóm kiểm thử, và kiểm chứng hệ thống bằng kiểm thử tự động.

0.3. Phạm vi

Hệ thống hướng tới nhóm nhỏ (khoảng 2–15 người) làm dự án phần mềm. Đây là ứng dụng web dùng được trên trình duyệt máy tính và điện thoại, bằng tiếng Anh và tiếng Việt. Ngoài phạm vi: chấm công, thanh toán, biểu đồ Gantt, ứng dụng di động gốc và tích hợp với các hệ thống quản lý issue khác.

0.4. Phương pháp thực hiện

Nhóm áp dụng chính quy trình mà sản phẩm hỗ trợ. Công việc được tổ chức theo sprint với một backlog chung; mỗi thay đổi đi kèm kiểm thử, mọi quyết định quan trọng đều được ghi lại. Dự án duy trì tám tài liệu sống (D1 Yêu cầu, D2 Bản đồ module, D3 Kiến trúc, D4 Hợp đồng API và dữ liệu, D5 Kiểm thử và tiêu chí chấp nhận, D6 Rủi ro và chính sách, D7 Sổ tay phát triển và nhật ký quyết định, D8 Ma trận truy vết yêu cầu) nằm cùng kho mã nguồn, để mỗi thay đổi mã và tài liệu mô tả nó được xem xét cùng nhau. Kiểm thử theo hướng *characterisation-first*: trước khi sửa đoạn mã chưa có test, viết test mô tả hành vi hiện tại; bản thân các test được kiểm tra bằng *kiểm thử đột biến* (mutation testing) [2]—cố ý làm hỏng mã và xác nhận có test báo lỗi.

0.5. Tổ chức nhóm và phân công nhiệm vụ

Nhóm gồm ba thành viên lớp Kỹ thuật máy tính K23A. Bảng 1 trình bày phân công công việc.

Bảng 1. *Phân công nhiệm vụ các thành viên*

Thành viên	**Vai trò**	**Nhiệm vụ**
Phạm Văn Huynh (KTMT K23A)	Nhóm trưởng Scrum Master	• Lập kế hoạch dự án, tổ chức sprint, theo dõi tiến độ • Phân tích yêu cầu và bộ tài liệu D1–D8 • Module Scrum: vòng đời sprint, lập kế hoạch backlog, burndown, velocity, retrospective AI • Tích hợp AI với Groq, thiết kế prompt; AI trả lời theo ngôn ngữ giao diện • Triển khai: Docker, MySQL, Cloudflare Tunnel trên `koshi.huynhca.com` • Dữ liệu kiểm thử, smoke test đầu-cuối, video demo, báo cáo
Phạm Minh Tú (KTMT K23A)	Lập trình viên chính (kiến trúc, frontend)	• Kiến trúc tổng thể và lựa chọn công nghệ • Frontend Vue 3: Pinia store, Kanban và bảng, bộ xử lý phím tắt • Đồ thị phụ thuộc, sắp xếp topo và đường găng • Lưu trữ local-first (IndexedDB) và chính sách ghi khi ngoại tuyến • Trang giới thiệu, trang hồ sơ, đa ngôn ngữ Anh/Việt

Thành viên	Vai trò	Nhiệm vụ
Đàm Đức Đôn (KTMT K23A)	Lập trình viên (backend, chất lượng)	• Backend FastAPI: xác thực, dự án, vai trò theo dự án và lời mời • Bình luận, @mention, trả lời, đính kèm, thông báo, lịch sử task • Lược đồ CSDL, Alembic migration, chuyển từ SQLite sang MySQL • Tích hợp liên tục (GitHub Actions), bộ test backend và frontend • Logic dự phòng AI và xoay vòng API key

0.6. Cấu trúc báo cáo

Ngoài phần mở đầu và kết luận, báo cáo gồm năm chương. Chương 1 trình bày cơ sở lý thuyết: Kanban, Scrum, đồ thị phụ thuộc và đường găng, phần mềm local-first, mô hình ngôn ngữ lớn và các công nghệ sử dụng. Chương 2 phân tích yêu cầu. Chương 3 mô tả thiết kế hệ thống: kiến trúc, mô hình dữ liệu, API, chuỗi dự phòng AI, bảo mật và chính sách ngoại tuyến. Chương 4 trình bày kết quả cài đặt kèm ảnh chụp màn hình, chiến lược kiểm thử, kết quả và việc triển khai. Chương 5 mô tả "nhà máy phần mềm" agent AI dùng để xây dựng hệ thống: kỹ thuật ngữ cảnh, các trạm của dây chuyền, rào chắn thực thi bằng mã, kiểm thử đơn vị và tích hợp, cùng việc đóng gói, vận hành và theo dõi sản phẩm.

CHƯƠNG 1: CƠ SỞ LÝ THUYẾT

Chương này trình bày các phương pháp quản lý dự án mà hệ thống hỗ trợ, các thuật toán phía sau chức năng phụ thuộc, và các công nghệ dùng để xây dựng hệ thống.

1.1. Quản lý dự án cho nhóm phần mềm nhỏ

1.1.1. Kanban

Kanban trực quan hoá công việc dưới dạng thẻ di chuyển qua các cột biểu diễn trạng thái, và giới hạn lượng việc đang làm để nhóm hoàn thành việc trước khi bắt đầu việc mới [3]. Ưu điểm là luồng công việc liên tục: không có vòng lặp cố định, và việc bị chặn hiện ra ngay. Hệ thống dùng quy trình bốn trạng thái cố định:

$$\text{TODO} \to \text{IN_PROGRESS} \to \text{BLOCKED} \to \text{DONE} \to \text{TODO}.$$

Vòng trạng thái này được cài đặt ở cả client và server và phải luôn giống nhau; task ở trạng thái BLOCKED mang theo lý do, nên bảng công việc giải thích được *vì sao* việc bị dừng.

1.1.2. Scrum

Scrum là một khung làm việc trong đó nhóm bàn giao công việc theo các vòng lặp có độ dài cố định gọi là *sprint*, thường từ một đến bốn tuần [4]. Các tạo tác gồm *product backlog* (mọi việc có thể làm), *sprint backlog* (việc nhóm cam kết trong sprint này) và phần tăng trưởng sản phẩm. Các sự kiện gồm sprint planning, daily scrum, sprint review và *sprint retrospective*—nơi nhóm nhìn lại cách làm việc và quyết định cần thay đổi gì. Hai thước đo chuẩn:

- **Burndown**: khối lượng còn lại (story point) theo từng ngày của sprint, so với đường lý tưởng đi thẳng từ mức cam kết về 0.
- **Velocity**: số story point hoàn thành mỗi sprint, dùng để định lượng cam kết sprint sau cho thực tế.

Hệ thống hỗ trợ Scrum trên nền bảng Kanban: mỗi task hoặc nằm trong product backlog, hoặc thuộc một sprint; story point chính là độ phức tạp của task (S=1, M=2, L=3, XL=5).

1.2. Đồ thị phụ thuộc và đường găng

1.2.1. Sắp xếp topo

Nếu task A phụ thuộc task B thì B phải xong trước. Các task và phụ thuộc tạo thành đồ thị có hướng $G = (V, E)$ với cạnh $B \to A$. Khi G không có chu trình (*đồ thị có hướng không chu trình*, DAG), luôn tồn tại một *thứ tự topo*—dãy mà mỗi task đứng sau mọi điều kiện tiên quyết của nó—và có thể tìm bằng thuật toán Kahn [5]: lặp lại việc lấy một task không còn điều kiện tiên quyết chưa xong, đưa ra, rồi xoá các cạnh đi ra của nó. Khi nhiều task cùng sẵn sàng, hệ thống ưu tiên theo độ ưu tiên, rồi hạn chót, rồi thời điểm

tạo, để thứ tự gợi ý đặt việc khẩn nhất chưa bị chặn lên đầu. Chu trình (người dùng có thể vô tình tạo ra) không làm thuật toán lỗi: các task còn lại trong chu trình được đưa xuống cuối.

1.2.2. Đường găng

Phương pháp đường găng (CPM) [6] tìm chuỗi hoạt động phụ thuộc dài nhất; mọi chậm trễ trên chuỗi này làm chậm cả dự án. Hệ thống tính chuỗi nặng nhất gồm các task *chưa xong* bằng tìm kiếm theo chiều sâu có ghi nhớ, với trọng số của task t là

$$w(t) = p(t) \cdot c(t), \qquad p \in \{1,2,5,10\} \text{ (LOW..CRITICAL)}, \quad c \in \{1,3,5,8\} \text{ (S..XL)},$$

và trọng số của chuỗi là tổng trọng số các task. Task trên đường găng được đánh dấu trên bảng và trong màn hình DAG. Một lỗi tinh vi phát hiện trong dự án liên quan tới việc ghi nhớ khi gặp chu trình: giá trị tính cho một lượt duyệt bị cắt ngang bởi chu trình phụ thuộc vào đường đi, nên lưu cache khiến kết quả phụ thuộc thứ tự mảng đầu vào. Các kết quả như vậy giờ không bao giờ được cache.

1.3. Phần mềm local-first

Phần mềm local-first giữ bản chính của dữ liệu trên thiết bị người dùng và coi server là một đầu đồng bộ [1]. Lợi ích là phản hồi tức thì và chạy được khi ngoại tuyến; khó khăn chính là gộp các chỉnh sửa đồng thời. Hệ thống chọn chính sách thận trọng có chủ đích: trình duyệt ghi mọi thay đổi vào IndexedDB trước và hiển thị ngay, sau đó mới đồng bộ với server; khi ngoại tuyến, dự án *cá nhân* (một thành viên) vẫn sửa được, còn dự án *chung* chuyển sang chỉ đọc, vì khi chưa có thuật toán gộp, hai người cùng sửa ngoại tuyến sẽ âm thầm ghi đè lên nhau.

1.4. Mô hình ngôn ngữ lớn và đầu ra có cấu trúc

Các mô hình ngôn ngữ lớn dựa trên kiến trúc Transformer [7] có thể tóm tắt, trích xuất và lập kế hoạch từ ngôn ngữ tự nhiên. Tuy nhiên đầu ra của chúng là văn bản tự do, và ứng dụng không thể hành động dựa trên một đoạn văn. Vì vậy hệ thống yêu cầu mô hình trả JSON theo lược đồ cố định (ví dụ `{went_well, to_improve, action_items}`), loại bỏ khung Markdown, kiểm tra và chuẩn hoá kết quả (giá trị enum lạ được đổi về mặc định; phụ thuộc trỏ tới task không tồn tại bị loại bỏ), và chuyển sang câu trả lời tất định khi đầu ra không dùng được. Mô hình sử dụng là `openai/gpt-oss-120b` [8] chạy trên Groq qua API tương thích OpenAI [9].

1.5. Công nghệ sử dụng

Bảng 1.1. *Các công nghệ sử dụng*

Tầng	**Công nghệ**	**Mục đích**
Frontend	Vue 3, TypeScript, Pinia, Vite, Tailwind CSS	Ứng dụng một trang; component dùng Composition API và một store trung tâm [10]
Lưu trữ cục bộ	IndexedDB (idb-keyval)	Bản sao local-first các task của từng dự án
Backend	Python, FastAPI, Pydantic v2	REST API với lược đồ request/response có kiểu [11]
Dữ liệu	SQLAlchemy 2, Alembic, MySQL 8.4	ORM, migration có phiên bản, CSDL production (SQLite khi phát triển)
Bảo mật	JWT (HS256), bcrypt	Phiên không trạng thái [12], băm mật khẩu
AI	Groq API, Ollama (tuỳ chọn)	LLM trên cloud, LLM cục bộ tuỳ chọn
Kiểm thử	pytest, Vitest, Vue Test Utils, Playwright	Kiểm thử đơn vị, component, tự động hoá trình duyệt
Triển khai	Docker Compose, nginx, GitHub Actions, Cloudflare Tunnel	Container, reverse proxy, CI, truy cập công khai không cần mở cổng [13]

1.6. Tài liệu như một hệ thống kiểm soát

Tài liệu mô tả hệ thống sẽ sai ngay khi mã thay đổi mà tài liệu không đổi theo; dự án này kế thừa một bản đặc tả mô tả những hành vi mà mã không hề có. Vì vậy nhóm giữ tám tài liệu cạnh mã nguồn (D1–D8, liệt kê ở phần Mở đầu), với quy tắc: một thay đổi mã làm tài liệu sai thì chưa hoàn thành cho tới khi tài liệu được cập nhật trong cùng commit. D7 ghi lại mọi quyết định quan trọng (*DEC-nnn*) và mọi lỗi phát hiện được (*F-nn*) cùng lý do, còn D8 truy vết từng yêu cầu tới mã nguồn và test kiểm chứng nó. Các số liệu ở Chương 4 lấy từ những bản ghi này.

CHƯƠNG 2: KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU

Chương này khảo sát các công cụ hiện có, xác định tác nhân và nêu các yêu cầu chức năng, phi chức năng. Danh sách đầy đủ (89 yêu cầu chức năng và 12 yêu cầu phi chức năng; tổng số theo nhóm dưới đây bằng 89) nằm trong tài liệu D1; ở đây chỉ tóm tắt các yêu cầu quan trọng nhất.

2.1. Khảo sát các công cụ hiện có

Bảng 2.1. *So sánh với các công cụ phổ biến*

Tiêu chí	**Trello**	**Jira**	**Bảng tính**	**Hệ thống của nhóm**
Bảng Kanban	Có	Có	Thủ công	Có
Báo cáo Scrum (burndown, velocity)	Tiện ích thêm	Có	Không	Có
Phụ thuộc task, đường găng	Không	Một phần	Không	Có (DAG)
Điều khiển hoàn toàn bằng bàn phím	Hạn chế	Hạn chế	Có	Có
Dùng khi ngoại tuyến	Không	Không	Có	Dự án cá nhân
AI tóm tắt, biên bản, giao việc	Trả phí	Trả phí	Không	Có
Độ khó cho sinh viên	Thấp	Cao	Thấp	Thấp

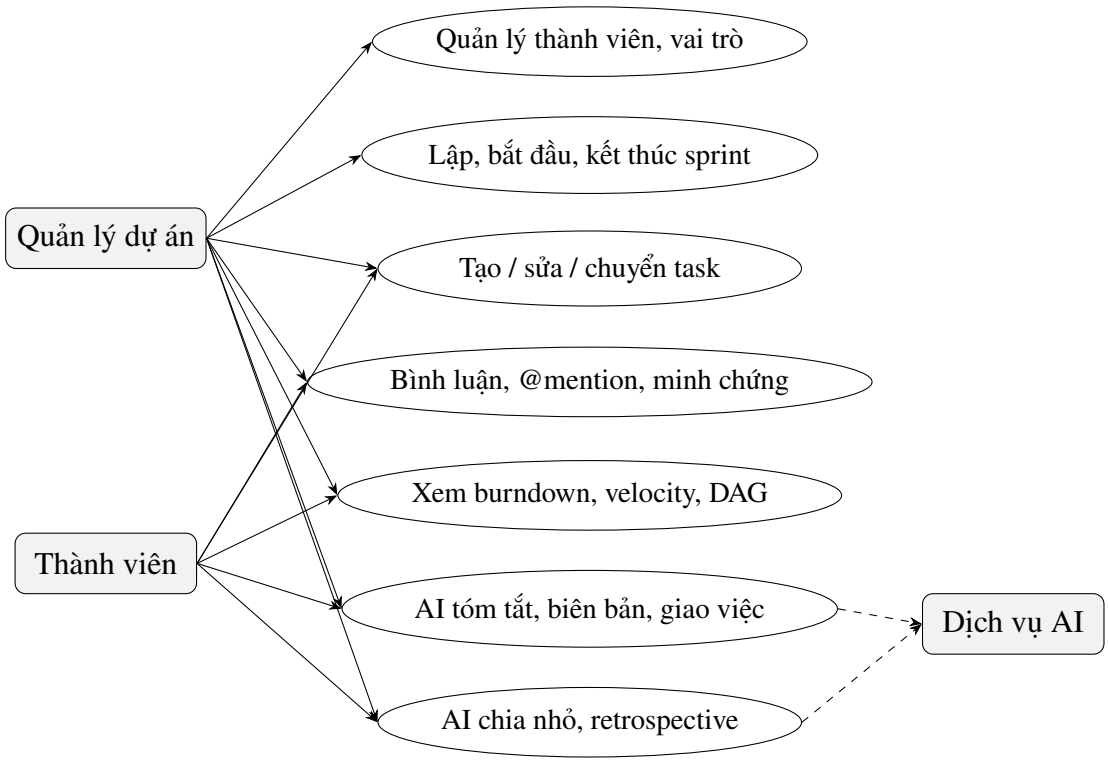
Khảo sát cho thấy khoảng trống mà dự án nhắm tới: một công cụ nhẹ như Trello nhưng vẫn có báo cáo Scrum, phân tích phụ thuộc và hỗ trợ AI, đồng thời dùng được khi không có mạng.

2.2. Tác nhân

Bảng 2.2. *Các tác nhân của hệ thống*

Tác nhân	**Mô tả**
Khách	Chưa đăng nhập. Xem trang giới thiệu (tính năng, video demo, bảng giá, hỏi đáp) và có thể đăng ký hoặc đăng nhập.
Người dùng	Mọi tài khoản đã đăng nhập. Bản thân tài khoản *không* có quyền gì: chỉ có thể tạo dự án, trả lời lời mời và sửa hồ sơ của mình.
Quản lý dự án (PM)	Người dùng giữ vai trò PM *trong một dự án cụ thể*. Quản lý thành viên và vai trò, lập kế hoạch và điều hành sprint. Người tạo dự án trở thành PM của dự án đó.
Thành viên	Người dùng giữ vai trò Thành viên trong dự án. Tạo và cập nhật task, bình luận, dùng công cụ AI; không được đổi vai trò hay sprint.
Dịch vụ AI	LLM bên ngoài (Groq) hoặc mô hình cục bộ (Ollama) trả lời các prompt có cấu trúc.

Vai trò được đặt theo từng dự án một cách có chủ đích: cùng một người có thể là PM ở dự án này và là Thành viên ở dự án khác. Khi đăng ký, hệ thống không bao giờ hỏi vai trò.



Hình 2.1. *Các ca sử dụng chính (PM làm được mọi việc của Thành viên)*

2.3. Phân tích nghiệp vụ

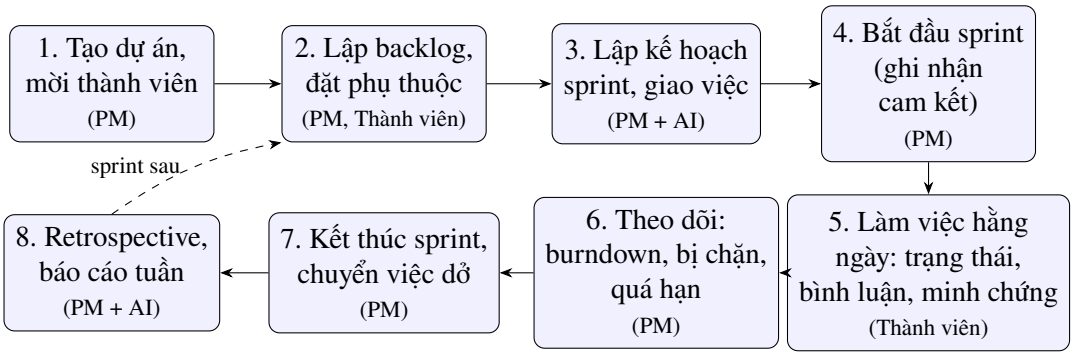
2.3.1. Quy trình hiện tại và các vấn đề

Một nhóm sinh viên thường phối hợp qua nhóm chat và một bảng tính dùng chung. Quy trình này có bốn vấn đề lặp lại:

- **Không có một nguồn sự thật duy nhất.** Việc được thống nhất trong chat rồi chép tay sang bảng tính; hai nơi lệch nhau và không ai biết bản nào mới nhất.
- **Vướng mắc không ai thấy.** Thành viên bị kẹt nhắn một tin rồi tin bị trôi; trưởng nhóm chỉ biết khi đến hạn.
- **Không có thứ tự.** Các việc phụ thuộc nhau được liệt kê ngang hàng, nên có người bắt đầu việc khi việc tiên quyết chưa xong.
- **Báo cáo thủ công.** Báo cáo tuần và biên bản họp viết theo trí nhớ; việc giao nhiệm vụ theo thói quen chứ không theo kỹ năng hay khối lượng.

2.3.2. Quy trình mục tiêu

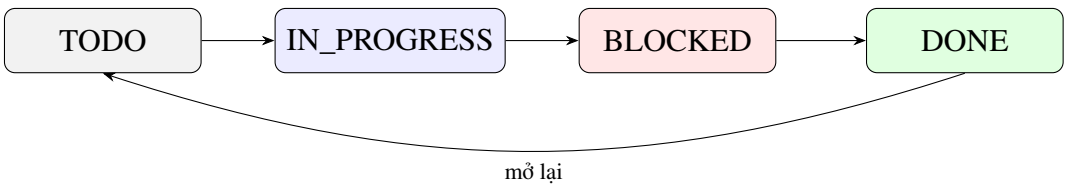
Hệ thống hỗ trợ chu trình theo kiểu Scrum ở Hình 2.2. Mỗi bước là một hoạt động nghiệp vụ do một vai trò thực hiện, và tạo ra dữ liệu cho bước sau.



Hình 2.2. *Quy trình nghiệp vụ mục tiêu của một nhóm dự án*

2.3.3. Vòng đời của task

Một task đi qua bốn trạng thái. Thứ tự cố định và giống nhau ở client và server:



Hình 2.3. *Vòng trạng thái task (phím Space chuyển một bước)*

2.3.4. Quy tắc nghiệp vụ

Bảng 2.3. *Các quy tắc nghiệp vụ chính*

Mã	**Quy tắc**
BR-01	Quyền hạn thuộc về vai trò *trong một dự án*; tài khoản tự thân không có quyền gì. Người tạo dự án là PM của dự án đó.
BR-02	Người được mời chỉ có quyền truy cập sau khi chấp nhận; lời mời đang chờ không cấp quyền gì.
BR-03	Chỉ thành viên thấy dự án; với người khác, dự án như không tồn tại (404).
BR-04	Chỉ PM quản lý thành viên, vai trò và sprint.
BR-05	Mỗi dự án có tối đa một sprint đang chạy; cam kết (story point) được chốt khi sprint bắt đầu.
BR-06	Khi kết thúc sprint, task chưa xong chuyển sang sprint sau hoặc backlog; xoá sprint không bao giờ xoá công việc.
BR-07	Task bị chặn nên ghi rõ lý do; task đã xong không bao giờ hiển thị là quá hạn.
BR-08	Mọi người được giao phải là thành viên đã chấp nhận của dự án; phụ thuộc chỉ trong cùng một dự án.
BR-09	Thứ tự trên bảng: trễ hạn trước, rồi hạn gần hơn, rồi đến độ ưu tiên.
BR-10	Dự án cá nhân sửa được khi mất mạng; dự án nhóm chỉ xem khi mất mạng để tránh ghi đè lẫn nhau.
BR-11	Không ai nhận thông báo về hành động của chính mình.

2.4. Yêu cầu chức năng

Bảng 2.4. *Các nhóm yêu cầu chức năng*

Nhóm	**Yêu cầu tiêu biểu**	**Số lượng**
FR-AUTH	Đăng ký bằng email và mật khẩu (không chọn vai trò); đăng nhập; phiên JWT; vai trò theo dự án; người ngoài nhận 404 với dự án họ không được xem.	10
FR-PROJ	Bảng điều khiển cá nhân; tạo, xoá dự án; mời qua email (chờ chấp nhận); đổi vai trò; trang hồ sơ có ảnh đại diện.	11
FR-DOM	Task có tiêu đề, mô tả, trạng thái, ưu tiên, độ phức tạp, hạn chót, lý do bị chặn, nhiều người thực hiện, phụ thuộc, tiêu chí chấp nhận; bình luận có @mention, trả lời một cấp, đính kèm; thông báo.	10

Nhóm	Yêu cầu tiêu biểu	Số lượng
FR-INT	Kanban và dạng bảng; mọi thao tác bằng phím ($h/j/k/l$, Space đổi trạng thái, n tạo mới, Enter xem chi tiết, / tìm kiếm, 1–4 đổi ưu tiên); sắp xếp theo hạn chót trước.	14
FR-GRAPH	Thứ tự topo, chịu được chu trình, đường găng hiển thị trên bảng và màn hình DAG.	5
FR-PERS	IndexedDB trước, server sau; dự án cá nhân sửa được khi ngoại tuyến, dự án chung chỉ đọc.	6
FR-SCRUM	Vòng đời sprint (đã lên kế hoạch, đang chạy, đã xong); mỗi dự án tối đa một sprint đang chạy; ghi nhận cam kết khi bắt đầu; chuyển việc chưa xong khi kết thúc; lập kế hoạch backlog; burndown; velocity; retrospective AI.	8
FR-HIST	Mọi thay đổi của task được ghi kèm người sửa và thời điểm; hiển thị trong màn hình chi tiết.	2
FR-AI	Tóm tắt tuần; biên bản họp; gợi ý giao việc; chia nhỏ mục tiêu; phân tích git diff; thống kê khối lượng và task trễ hạn; dự phòng ba tầng.	8
FR-NAV, FR-I18N, FR-MKT	Trang giới thiệu với nút đăng nhập góc trên bên phải; chuyển Anh/Việt cho toàn bộ giao diện; video demo quay từ ứng dụng thật.	15

2.4.1. Quy trình Scrum

Các yêu cầu Scrum định nghĩa một máy trạng thái nhỏ cho sprint:



Hình 2.4. *Vòng đời sprint*

Chỉ PM được thay đổi sprint; mỗi dự án tối đa một sprint đang chạy; sprint đã xong không nhận thêm việc; xoá sprint sẽ trả task về backlog chứ không bao giờ xoá công việc.

2.5. Yêu cầu phi chức năng

Bảng 2.5. *Các yêu cầu phi chức năng chính*

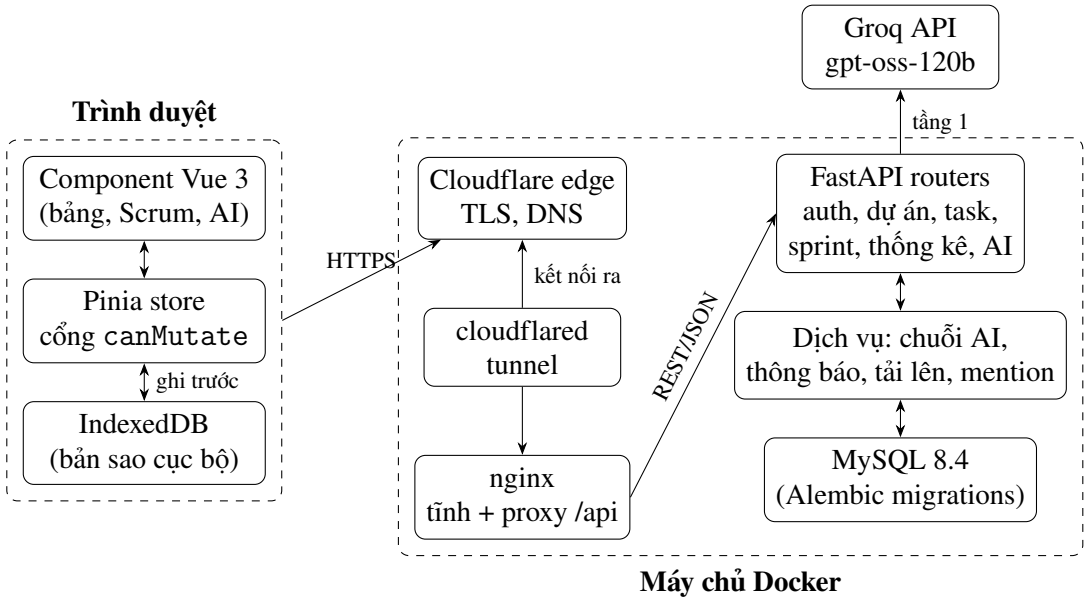
Thuộc tính	**Yêu cầu**
Độ phản hồi	Thay đổi cục bộ hiển thị ngay; lời gọi mạng không bao giờ chặn việc hiển thị.
Tính sẵn sàng của AI	Không endpoint AI nào trả lỗi vì mô hình không sẵn sàng; thay vào đó trả câu trả lời tất định và ghi log tình trạng suy giảm.
Bảo mật	Server kiểm tra quyền ở mọi route theo dự án; ngoài môi trường phát triển, dịch vụ từ chối khởi động nếu khoá bí mật yếu, còn tài khoản demo, CORS dạng wildcard hoặc lược đồ CSDL chưa cập nhật.
Toàn vẹn dữ liệu	Chỉ thay đổi lược đồ qua migration có phiên bản; bắt buộc khoá ngoại; UTF-8 (utf8mb4) xuyên suốt để tiếng Việt không bị cắt.
Đa ngôn ngữ	Mọi chuỗi hiển thị đều có bản tiếng Anh và tiếng Việt; câu trả lời của AI theo ngôn ngữ giao diện.
Khả năng kiểm thử	Bộ test backend và frontend chạy được trên bản checkout sạch và chạy trong CI.

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

Chương này mô tả kiến trúc, mô hình dữ liệu, API, và thiết kế ba vấn đề xuyên suốt định hình hệ thống nhiều nhất: phân quyền, chuỗi dự phòng AI và chính sách ngoại tuyến.

3.1. Kiến trúc tổng thể

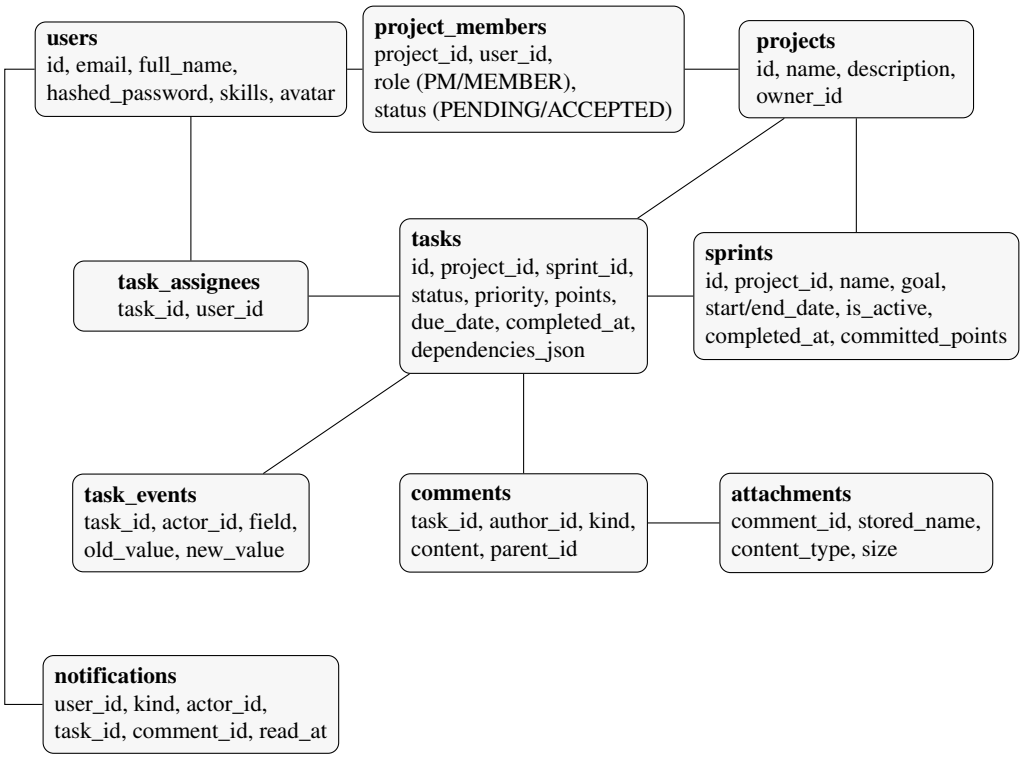
Hệ thống có kiến trúc client–server. Trình duyệt chạy ứng dụng một trang Vue, giữ bản sao cục bộ của dự án đang mở; một container nginx phục vụ ứng dụng và chuyển tiếp `/api` tới dịch vụ FastAPI, nơi sở hữu CSDL MySQL và gọi nhà cung cấp AI. Để truy cập công khai, Cloudflare Tunnel kết nối *từ máy chủ ra* mạng biên của Cloudflare, nên không phải mở cổng trên router và TLS do Cloudflare đảm nhận (Hình 3.1).



Hình 3.1. *Kiến trúc triển khai và các thành phần*

Frontend được tổ chức theo trách nhiệm: `components/` (26 component Vue), `stores/` (trạng thái task, giao diện, ngôn ngữ), `lib/` (các module thuần, không phụ thuộc framework: sắp xếp topo và đường găng, bộ xử lý phím tắt, sắp xếp theo độ gấp, phân tích git diff, phân tích mention, bản dịch) và `services/api.ts`, nơi duy nhất giao tiếp với server. Backend theo cùng cách tách: `routers/` (HTTP), `schemas/` (hợp đồng Pydantic), `models/` (thực thể SQLAlchemy), `services/` (AI, thông báo, tải lên) và `security.py` (xác thực và các hàm kiểm tra phạm vi dự án).

3.2. Mô hình dữ liệu



Hình 3.2. *Các thực thể (10 bảng)*

Ba quyết định thiết kế quan trọng nhất:

- **Quyền nằm ở tư cách thành viên, không nằm ở người dùng.** Bảng `users` không có cột vai trò; `project_members` giữ vai trò và trạng thái lời mời. Tư cách PENDING không cấp quyền gì: hàm tra cứu mà mọi hàm kiểm tra quyền sử dụng chỉ trả về khi đã chấp nhận.
- **Nhiều người thực hiện một task** qua bảng `task_assignees`; mỗi người phải là thành viên đã chấp nhận của dự án chứa task.
- **Lịch sử và Scrum cần thời gian.** `tasks.completed_at` được đặt khi task vào DONE và xoá khi rời DONE—đây là dữ liệu burndown đọc; cột `committed_points` của sprint lưu mức cam kết lúc bắt đầu—đây là mốc để velocity so sánh. `task_events` lưu chuỗi hiển thị thay vì khoá ngoại, nên một bản ghi vẫn đọc đúng sau khi người hay sprint mà nó nhắc tới bị đổi tên hoặc xoá.

Lược đồ chỉ được tạo và thay đổi bởi chín migration Alembic; ngoài môi trường phát triển, dịch vụ từ chối khởi động nếu CSDL chưa ở phiên bản mới nhất.

3.3. Thiết kế API

REST API có 51 endpoint dưới `/api`; Bảng 3.1 tóm tắt các endpoint.

Bảng 3.1. *Tổng quan API*

Router	**Số endpoint**	**Chức năng chính**
`/auth`	4	đăng ký, đăng nhập, đăng nhập Google, người dùng hiện tại
`/users`	5	danh sách kèm khối lượng, sửa hồ sơ của mình, tải lên/xem/xoá ảnh đại diện
`/projects`	11	dự án của tôi, tạo, xoá, thành viên, mời, chấp nhận/từ chối, đổi vai trò, xoá thành viên
`/tasks`	11	liệt kê/lọc, tạo, sửa, xoá, đổi trạng thái, bình luận, đính kèm, lịch sử
`/sprints`	9	liệt kê, tạo, sửa, bắt đầu, kết thúc, xoá, thống kê, burndown, velocity
`/stats`	2	khối lượng nhóm, task trễ hạn
`/notifications`	4	liệt kê, số chưa đọc, đánh dấu đã đọc một / tất cả
`/ai`	5	tóm tắt tuần, biên bản họp, giao việc, chia nhỏ, retrospective sprint

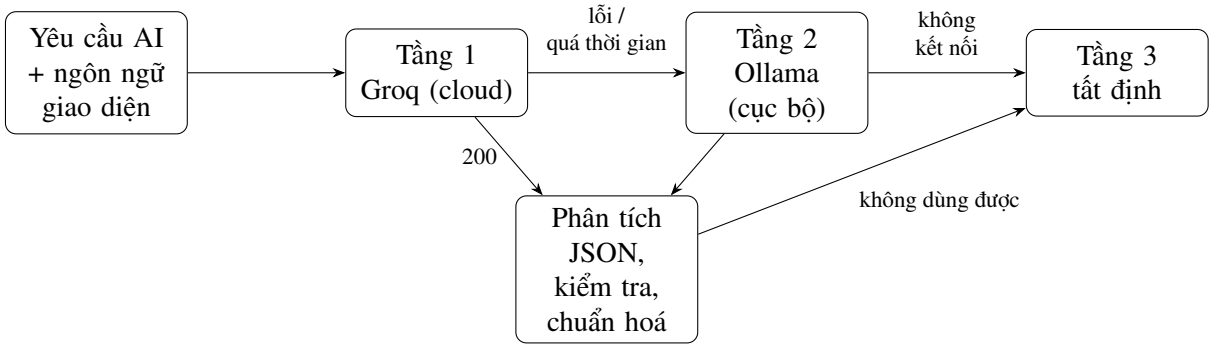
Mã lỗi theo ngữ nghĩa HTTP: 400 cho tham chiếu không hợp lệ (phụ thuộc không tồn tại, sprint của dự án khác), 403 khi thành viên thực hiện thao tác của PM, 404 với dự án người gọi không được xem—*không* phải 403, vì để lộ việc dự án tồn tại cũng đã là rò rỉ thông tin—và 409 khi bắt đầu sprint thứ hai trong lúc đã có sprint đang chạy.

3.4. Thiết kế phân quyền

Mọi route theo phạm vi dự án đều gọi một trong hai hàm kiểm tra, `require_member` hoặc `require_project_pm`, trước khi chạm vào dữ liệu. Giao diện ẩn các nút mà thành viên không được dùng, nhưng đó chỉ là gợi ý thao tác: server mới là ranh giới bảo mật [14]. Trong dự án, có một thay đổi đề xuất vào nhánh chính khiến client *giả làm* PM sau khi server đã từ chối đổi vai trò; thay đổi này bị loại chính vì lý do đó. Phiên đăng nhập là JWT ký bằng khoá bí mật riêng cho từng bản triển khai, được sinh trên máy chủ và không bao giờ được commit hay đóng gói vào image; bước kiểm tra khi khởi động từ chối khoá yếu hoặc khoá mặc định.

3.5. Chuỗi dự phòng AI

Mỗi chức năng AI gọi một hàm duy nhất, hàm này thử ba tầng theo thứ tự (Hình 3.3).



Hình 3.3. *Chuỗi dự phòng AI ba tầng*

Tầng tất định dựng câu trả lời từ chính dữ liệu dự án (ví dụ, retrospective đếm điểm cam kết và điểm hoàn thành, liệt kê task bị chặn kèm lý do, và phát hiện phạm vi bị thêm sau khi sprint bắt đầu), nên chức năng vẫn chạy khi không có mô hình nào. Việc rơi xuống tầng 3 được ghi log như một sự suy giảm; điều này quan trọng vì các tầng trả về cùng một dạng dữ liệu, nếu không thì một API key hết hạn trông vẫn hoàn toàn bình thường. Ngôn ngữ giao diện được gửi kèm mỗi yêu cầu (`Accept-Language`) và thêm vào prompt, nên giao diện tiếng Anh nhận câu trả lời tiếng Anh dù task được viết bằng tiếng Việt.

3.6. Chính sách ngoại tuyến và đồng bộ

Store áp dụng mọi thay đổi lên màn hình và IndexedDB trước, sau đó mới gửi lên server. Một thuộc tính duy nhất, `canMutate`, quyết định có được ghi hay không: luôn được khi có mạng; khi ngoại tuyến chỉ được ở dự án cá nhân. Mọi nút có thao tác ghi—kể cả công cụ AI chèn task—đều tự kiểm tra thuộc tính này, vì store từ chối một cách im lặng, và một nút không kiểm tra sẽ báo thành công dù không ghi gì. Hai lỗi kiểu này đã được phát hiện và sửa trong quá trình kiểm thử.

3.7. Thiết kế giao diện

Giao diện có ba màn hình, không dùng router: trang giới thiệu cho khách, bảng công việc và trang hồ sơ. Các hộp thoại cung cấp màn hình chi tiết task, bảng Scrum và các công cụ AI. Mọi chữ hiển thị đều lấy từ bảng dịch với một mục tiếng Anh và một mục tiếng Việt cho mỗi khoá (hơn 500 khoá); thiếu khoá tiếng Việt là lỗi biên dịch, và một test tự động sẽ báo lỗi nếu component chứa chữ viết cứng.

CHƯƠNG 4: TRIỂN KHAI VÀ KIỂM THỬ

Chương này trình bày hệ thống đã cài đặt qua ảnh chụp từ ứng dụng đang chạy, sau đó là chiến lược kiểm thử, kết quả và việc triển khai.

4.1. Môi trường phát triển

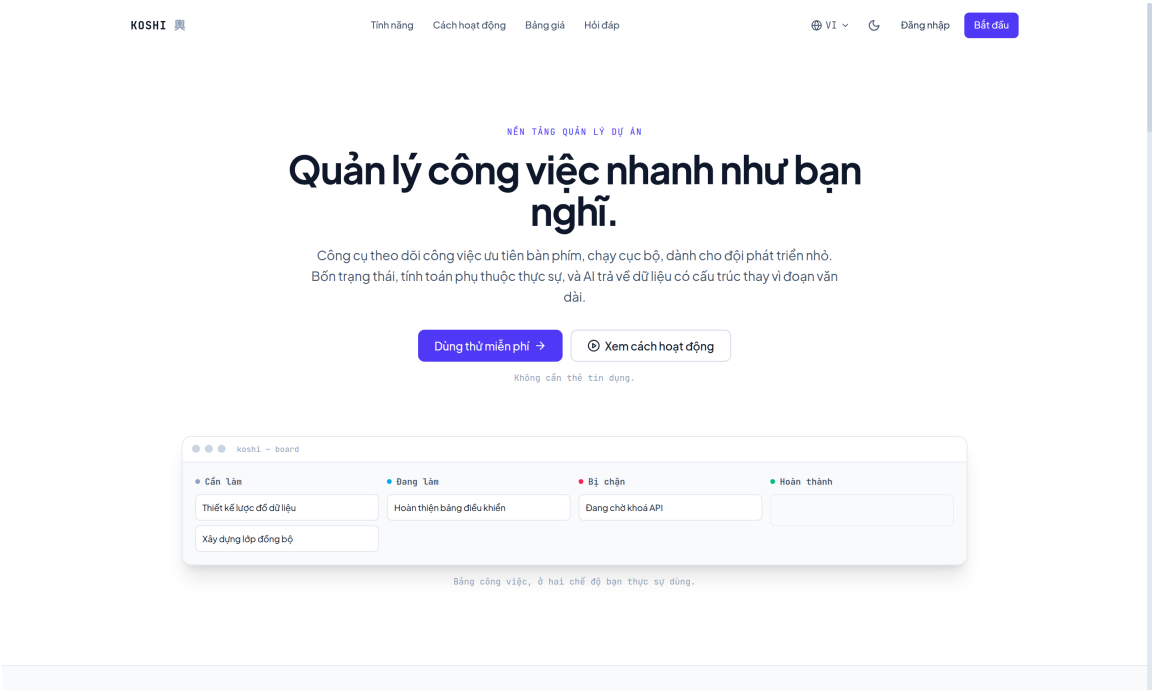
Bảng 4.1. *Môi trường phát triển và quy mô dự án*

Hệ điều hành	Ubuntu Linux, Docker Engine với Compose v2
Frontend	Node.js 22, pnpm 10, Vite, Vue 3, TypeScript
Backend	Python 3.12, FastAPI, SQLAlchemy 2, Alembic
Cơ sở dữ liệu	MySQL 8.4 (production), SQLite (phát triển, kiểm thử)
Mã nguồn	khoảng 13.300 dòng frontend, 3.400 dòng back-end (không tính dòng trống)
Mã kiểm thử	khoảng 5.200 dòng frontend, 2.800 dòng back-end
Kho mã	65 commit; 26 component Vue; 51 endpoint API; 10 bảng; 9 migration

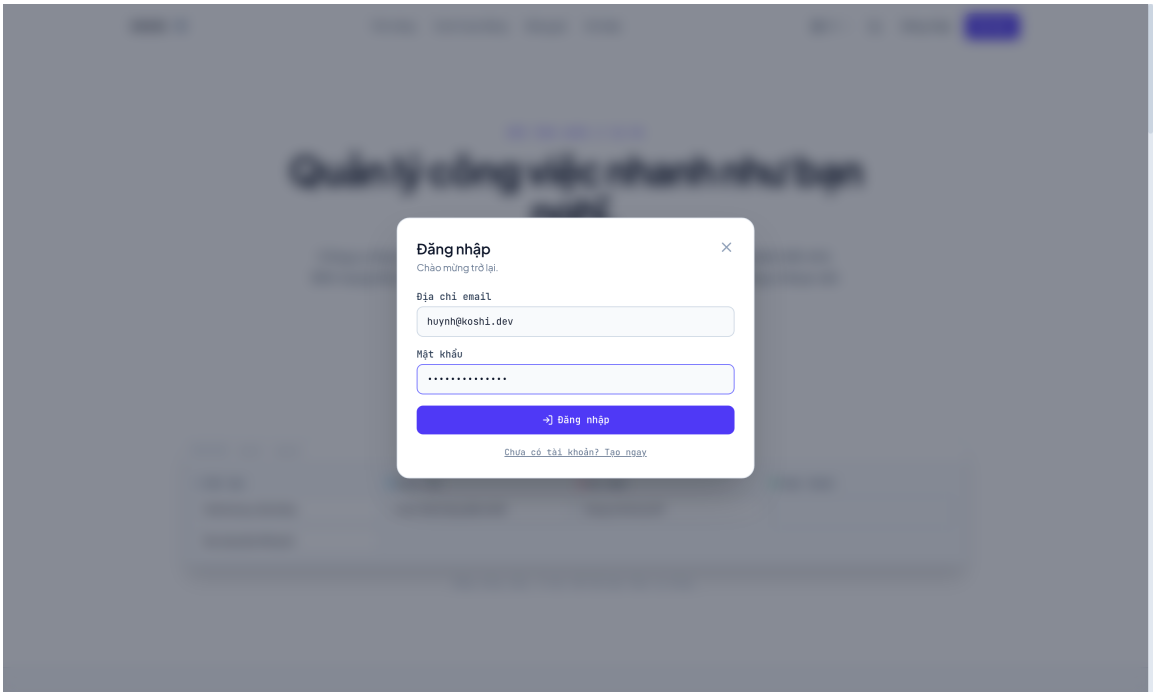
4.2. Các chức năng đã cài đặt

4.2.1. Trang giới thiệu và xác thực

Khách truy cập trước hết thấy trang giới thiệu với các tính năng, video demo và bảng giá; nút đăng nhập nằm ở góc trên bên phải và có thể chuyển ngôn ngữ Anh/Việt (Hình 4.1). Video demo là bản quay màn hình thật: một script điều khiển ứng dụng thật trên trình duyệt và ghi lại, nên nó chỉ có thể cho thấy những chức năng thực sự chạy. Khi đăng ký chỉ cần tên, email và mật khẩu—không bao giờ hỏi vai trò (Hình 4.2).



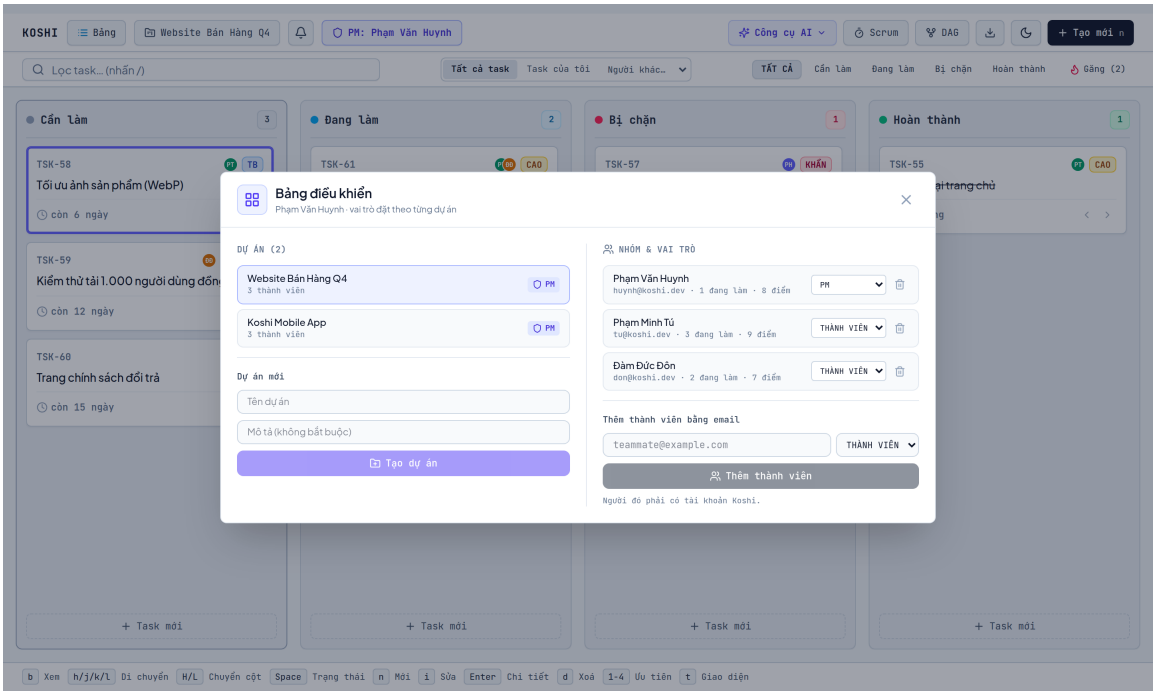
Hình 4.1. *Trang giới thiệu*



Hình 4.2. *Hộp thoại đăng nhập*

4.2.2. Dự án, thành viên và vai trò

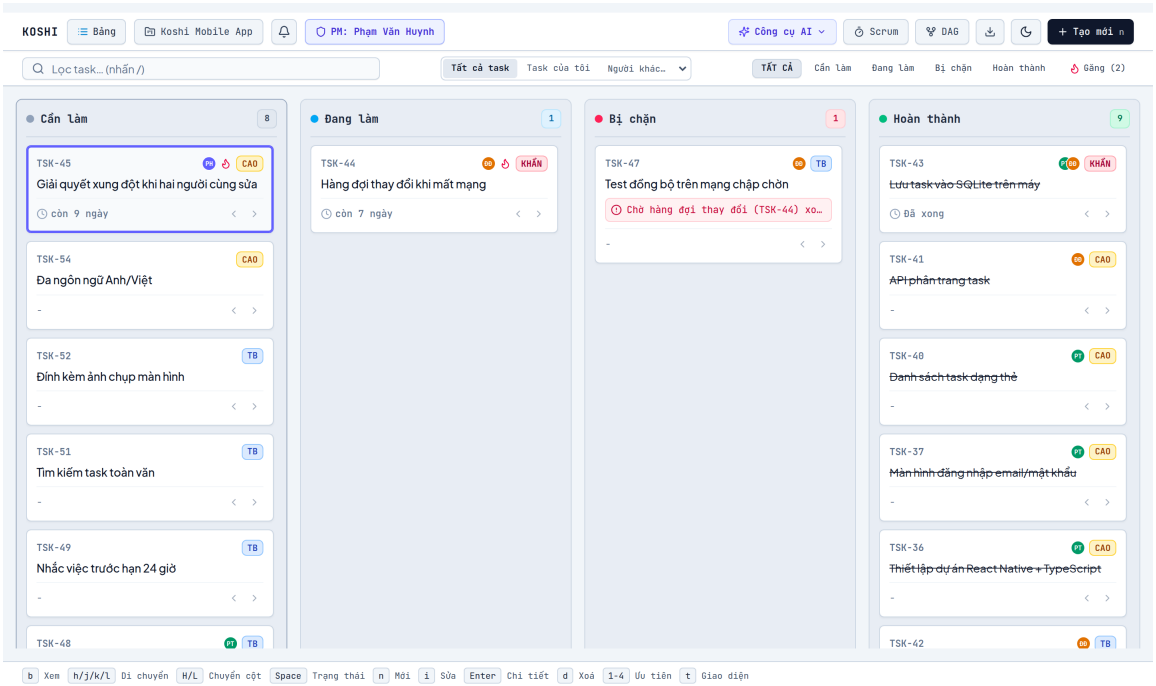
Bảng điều khiển liệt kê các dự án của người dùng kèm vai trò trong từng dự án, cho phép tạo dự án và quản lý nhóm (Hình 4.3). Thêm thành viên qua email sẽ gửi lời mời; người đó chỉ tham gia sau khi chấp nhận.



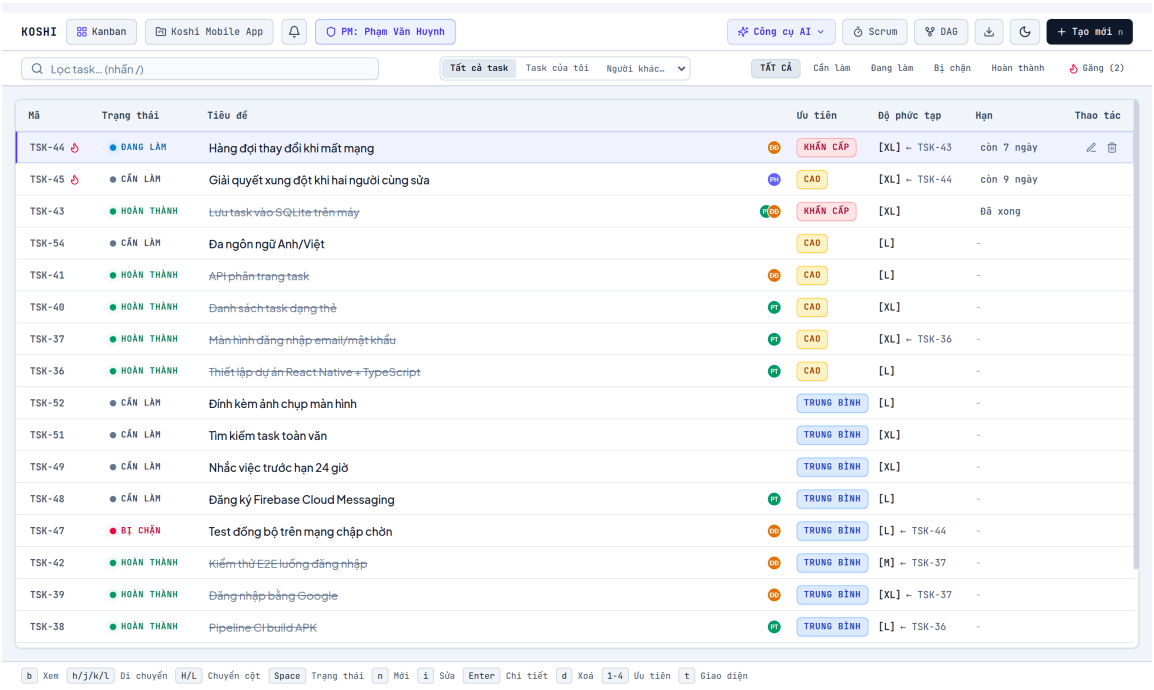
Hình 4.3. *Bảng điều khiển dự án với vai trò theo từng dự án*

4.2.3. Bảng Kanban và dạng bảng

Bảng có bốn cột. Mỗi thẻ hiển thị mã, tiêu đề, độ ưu tiên, ảnh đại diện người thực hiện, hạn chót (“còn 9 ngày”, “trễ 3 ngày”), lý do bị chặn và biểu tượng ngọn lửa cho task trên đường găng. Thứ tự sắp xếp ưu tiên hạn chót: một task ưu tiên thấp hết hạn hôm qua đứng trên một task khẩn cấp hạn tháng sau. Cùng dữ liệu đó có thể xem dạng bảng (Hình 4.4 và 4.5); phím b để chuyển đổi.

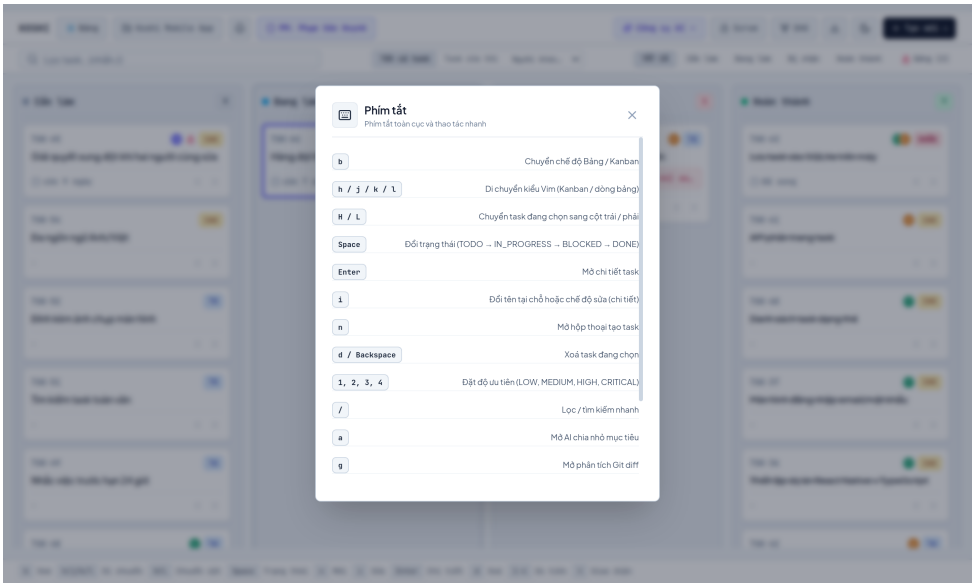


Hình 4.4. *Bảng Kanban*



Hình 4.5. *Dạng bảng*

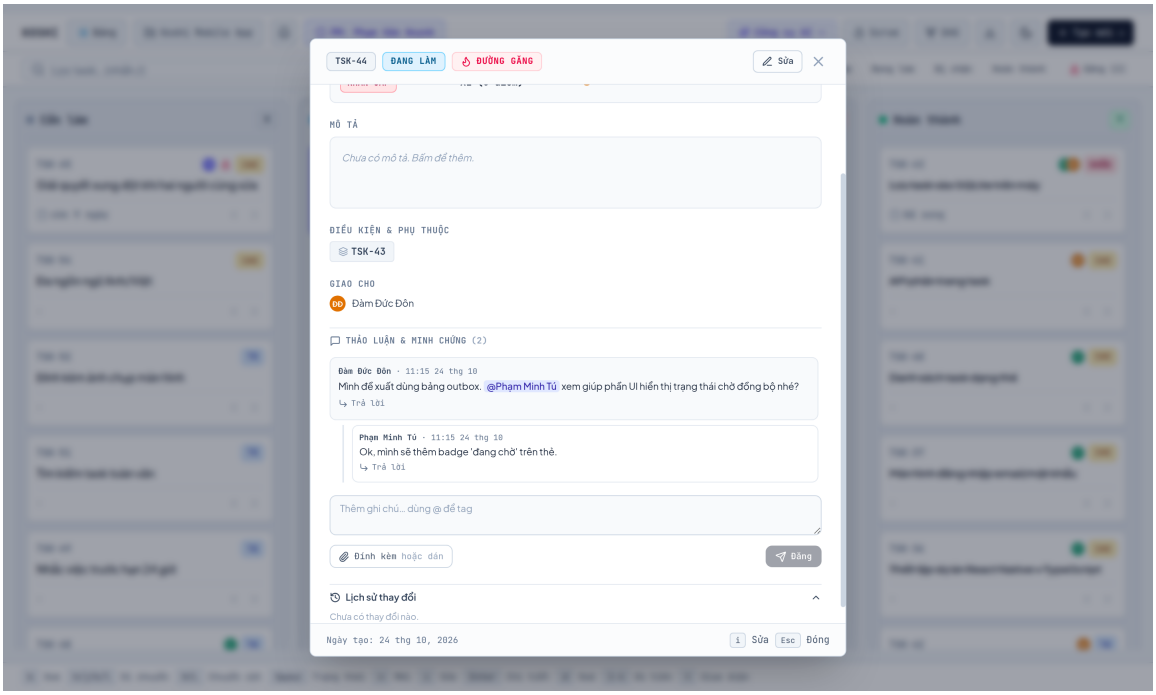
Mọi thao tác đều có phím tắt (Hình 4.6): $h/j/k/l$ di chuyển, Space đổi trạng thái, H/L chuyển task giữa các cột, 1–4 đặt độ ưu tiên, n tạo task và $/$ tìm kiếm.



Hình 4.6. *Phím tắt*

4.2.4. Chi tiết task: thảo luận, minh chứng và lịch sử

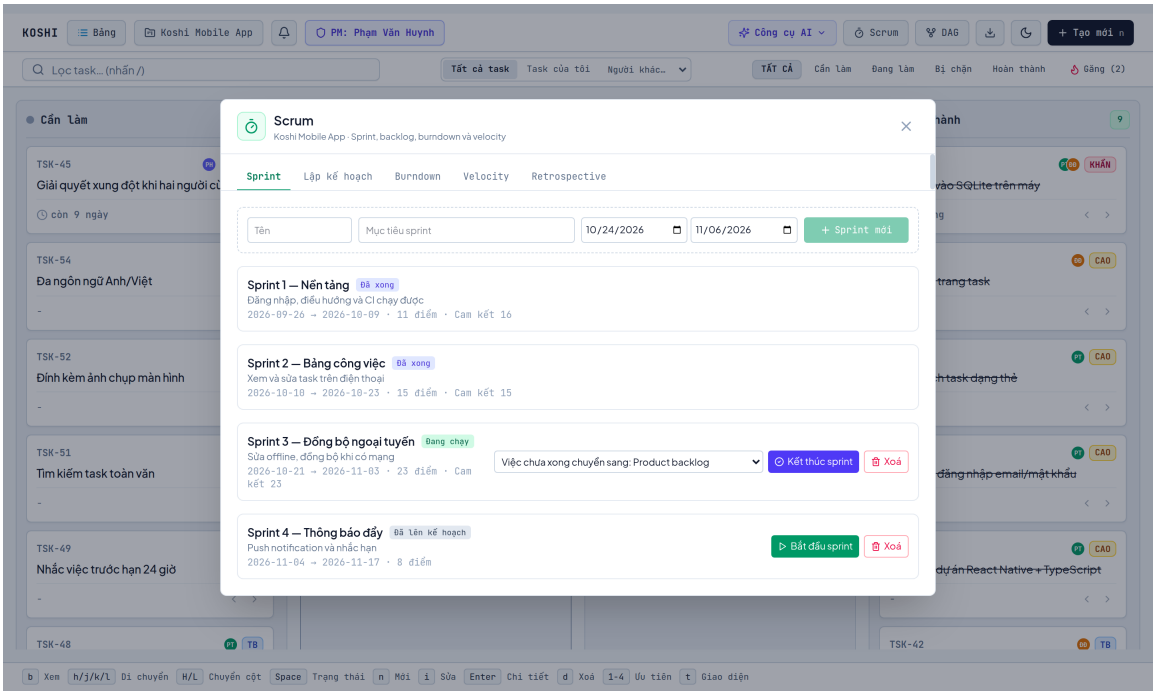
Mở một task sẽ thấy các trường, phụ thuộc và tiêu chí chấp nhận, luồng thảo luận có @mention và trả lời một cấp, file đính kèm, và lịch sử thay đổi do server ghi lại (Hình 4.7). Khi chuyển task sang DONE, hệ thống mời người dùng đính kèm minh chứng rằng việc đã xong.



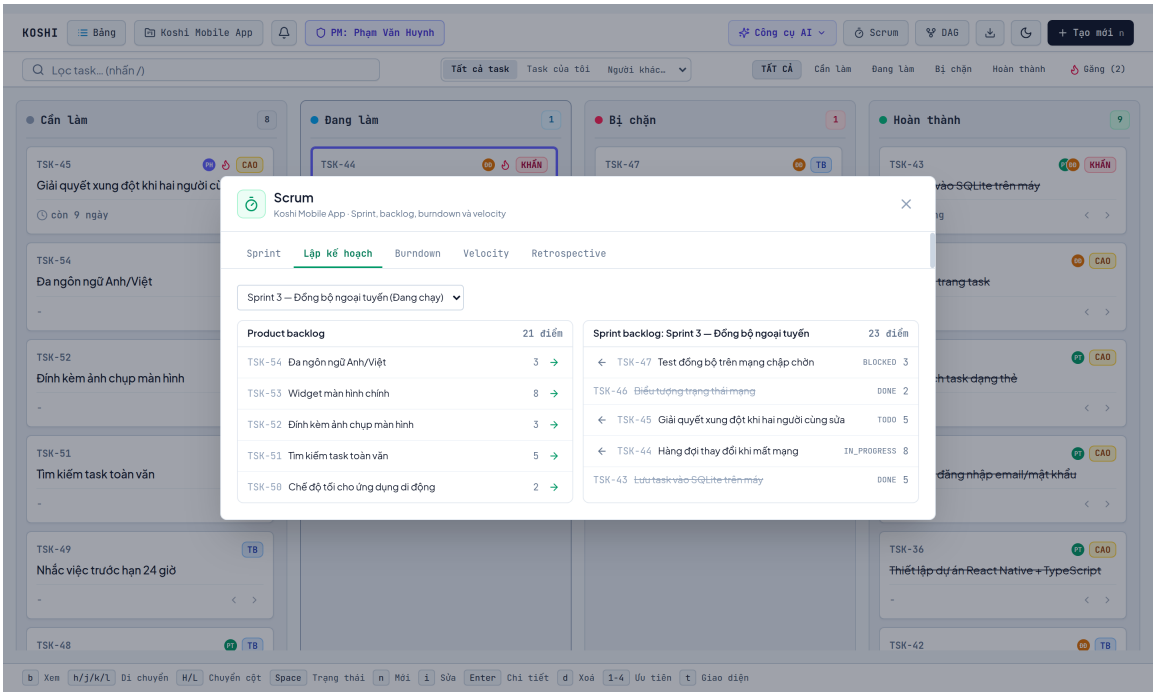
Hình 4.7. *Chi tiết task với thảo luận và lịch sử*

4.2.5. Scrum

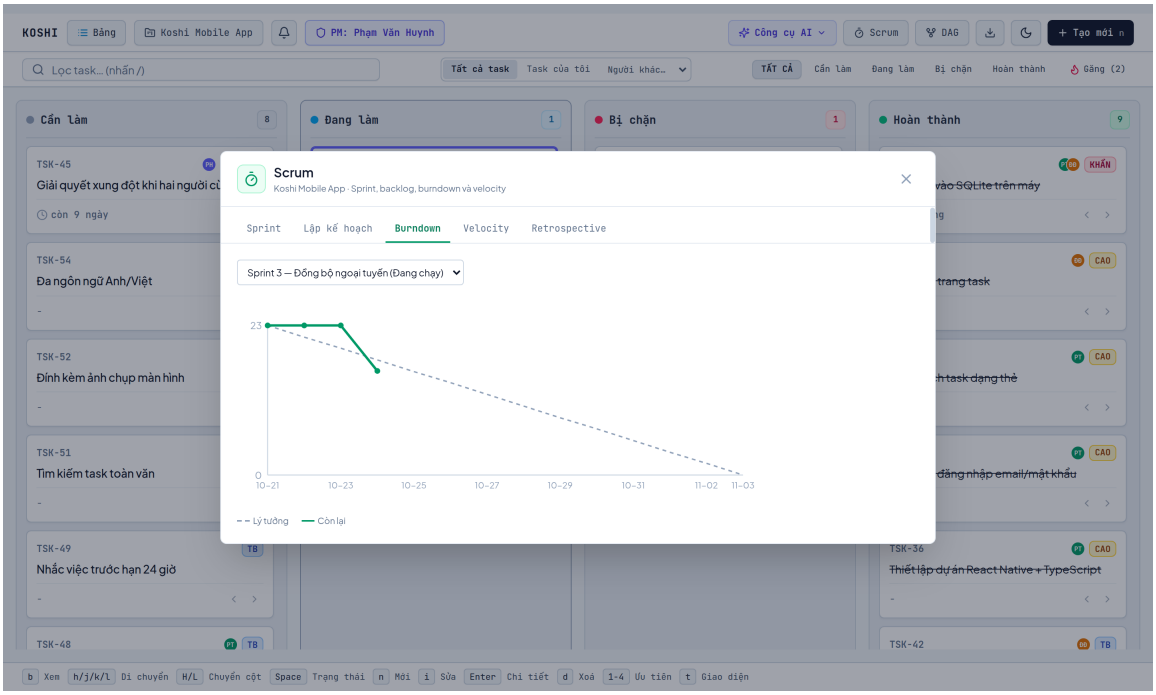
Bảng Scrum có năm tab. *Sprint* (Hình 4.8) liệt kê các sprint đã lên kế hoạch, đang chạy và đã xong cùng số điểm và mức cam kết; PM tạo, bắt đầu và kết thúc sprint, chọn nơi chuyển việc chưa xong. *Lập kế hoạch* (Hình 4.9) chuyển việc giữa product backlog và sprint. *Burndown* (Hình 4.10) vẽ điểm còn lại so với đường lý tưởng; *Velocity* (Hình 4.11) so sánh điểm cam kết và điểm hoàn thành của từng sprint cùng giá trị trung bình; *Retrospective* (Hình 4.12) nhờ AI nêu điều làm tốt, điều cần cải thiện và hành động cụ thể, dựa trên số liệu của sprint.



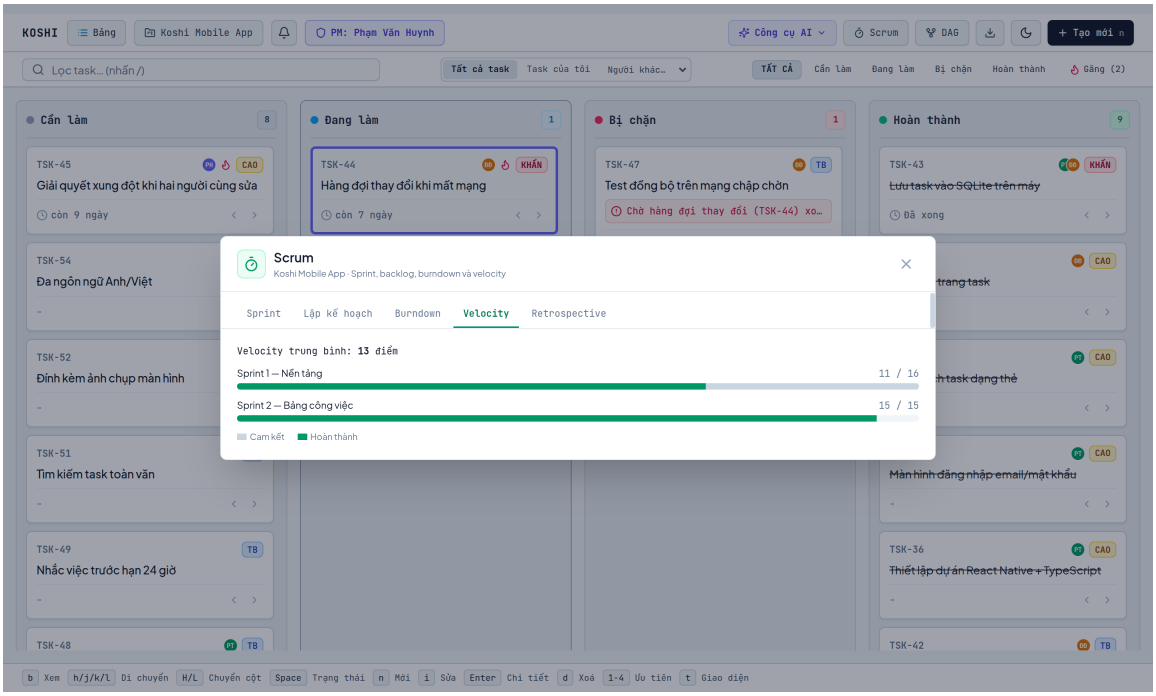
Hình 4.8. *Bảng Scrum: vòng đời sprint*



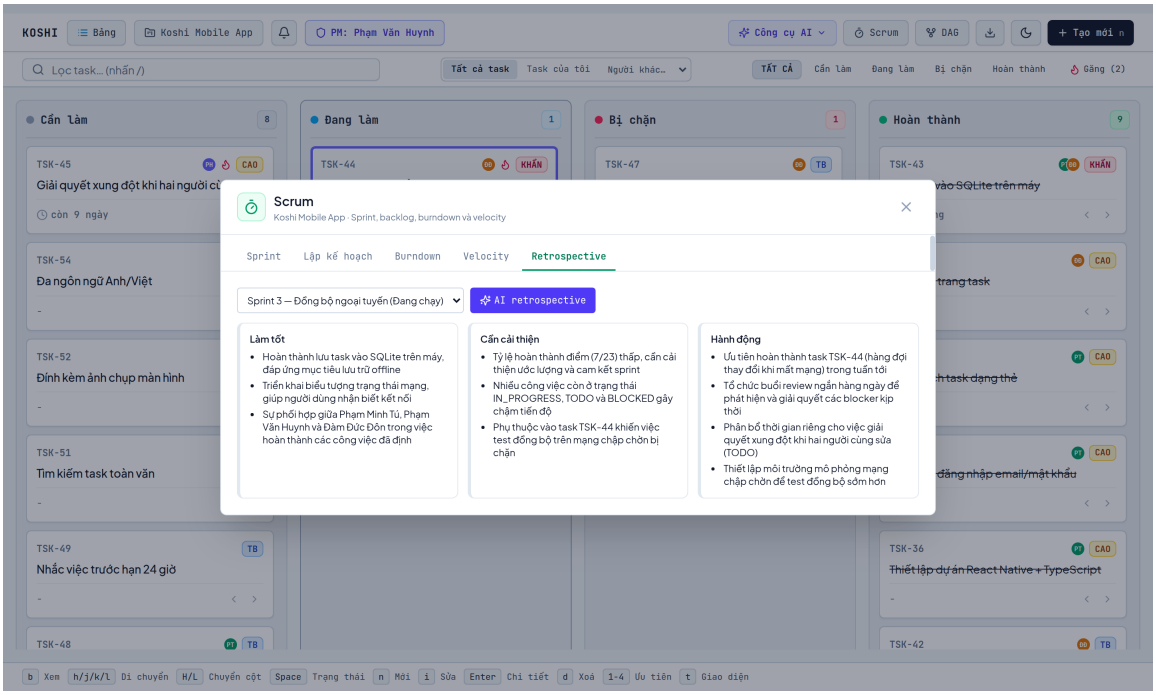
Hình 4.9. *Bảng Scrum: lập kế hoạch backlog*



Hình 4.10. *Bảng Scrum: biểu đồ burndown*



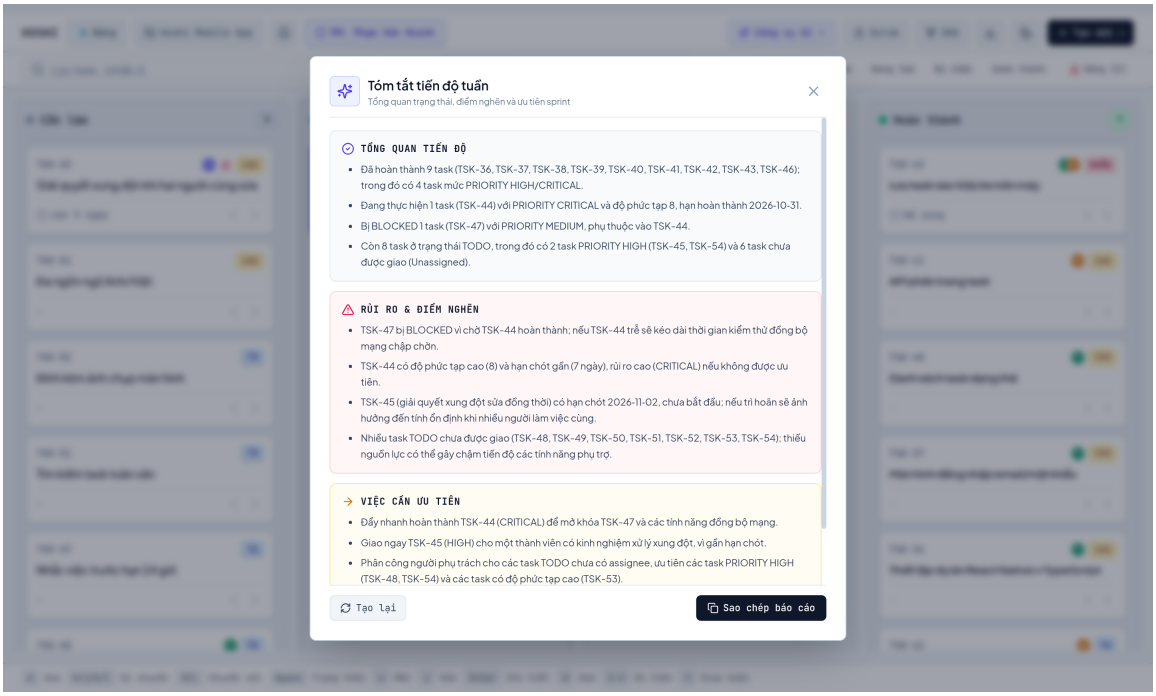
Hình 4.11. *Bảng Scrum: velocity*



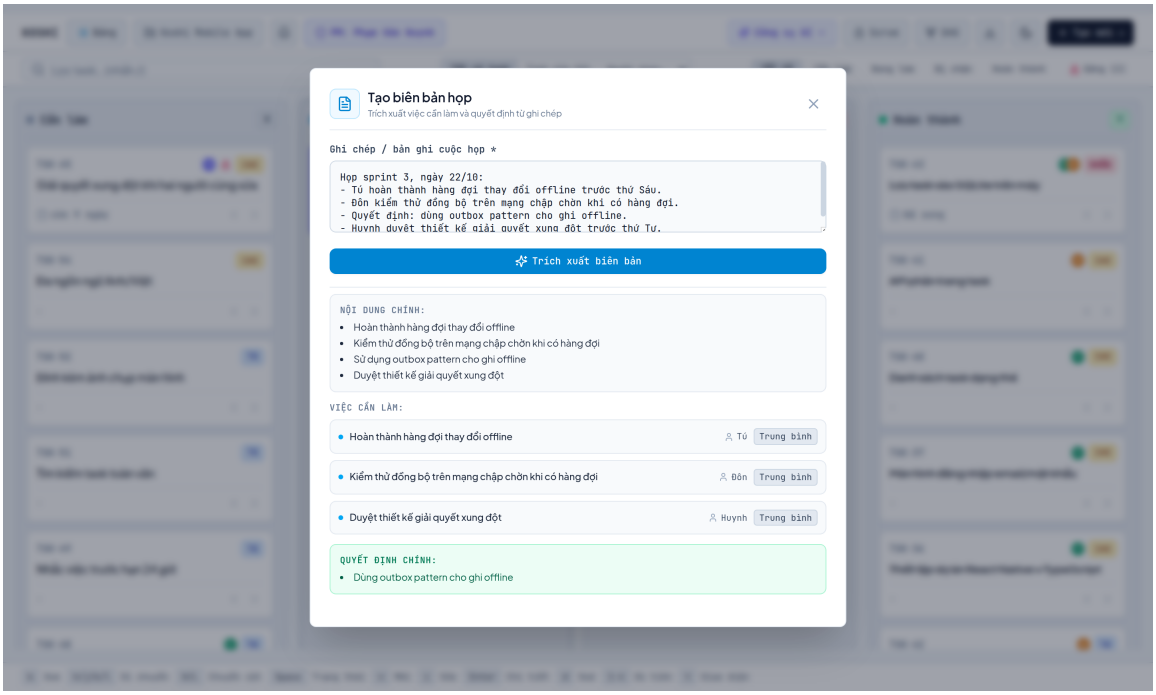
Hình 4.12. *Bảng Scrum: retrospective do AI viết*

4.2.6. Các công cụ AI

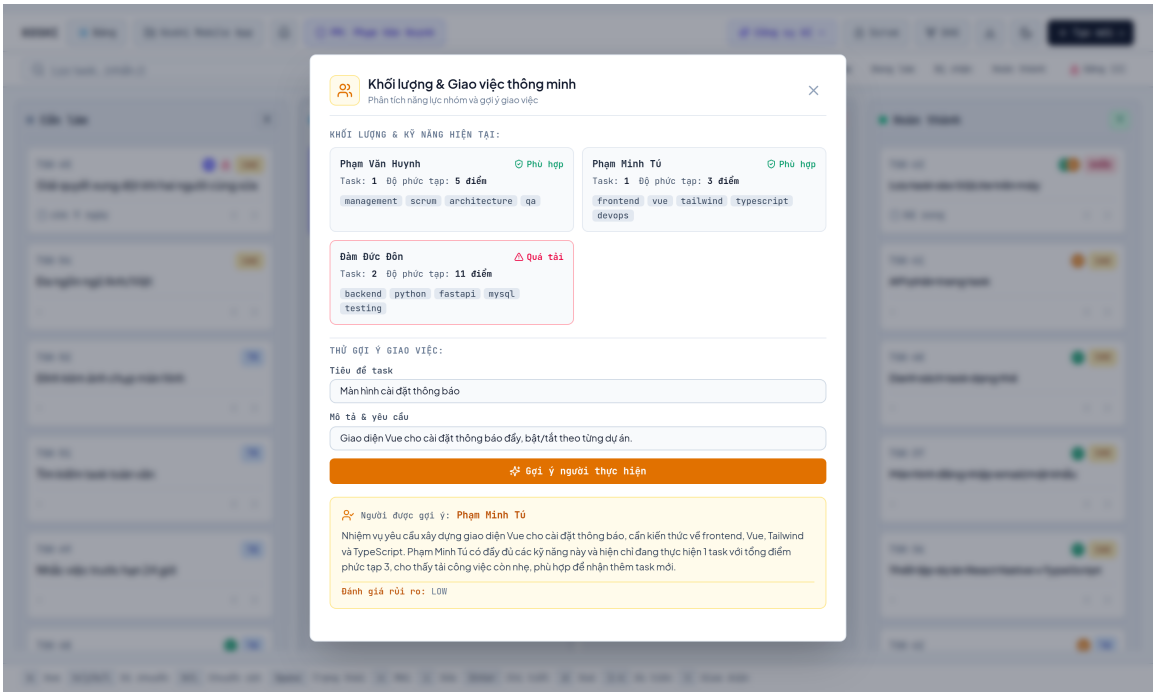
Bốn công cụ AI khác nằm trong menu *Công cụ AI*. Tóm tắt tuần báo cáo tiến độ, rủi ro và ưu tiên (Hình 4.13); trình tạo biên bản họp biến ghi chép thô thành chủ đề, việc cần làm có người phụ trách và độ ưu tiên, cùng các quyết định (Hình 4.14); giao việc thông minh hiển thị khối lượng và kỹ năng từng thành viên rồi gợi ý người thực hiện kèm lý do và đánh giá rủi ro (Hình 4.15); công cụ chia nhỏ mục tiêu tách một mục tiêu thành các task con có phụ thuộc, có thể chèn vào bảng bằng một thao tác (Hình 4.16).



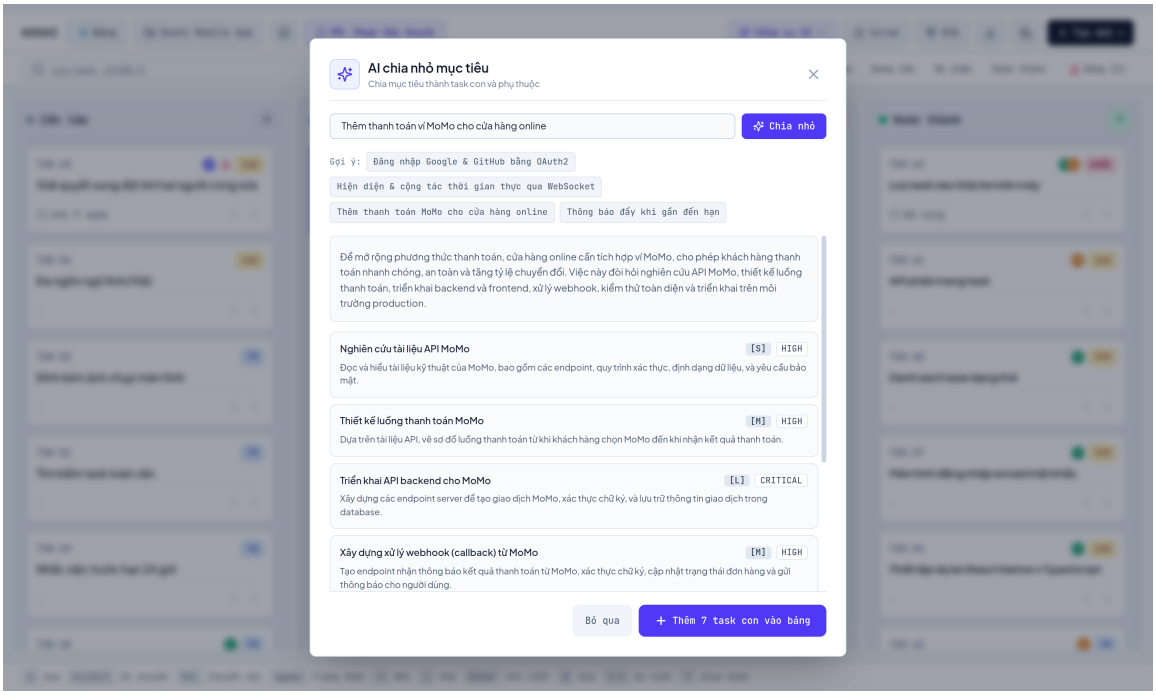
Hình 4.13. *AI tóm tắt tiến độ tuần*



Hình 4.14. *AI tạo biên bản họp*



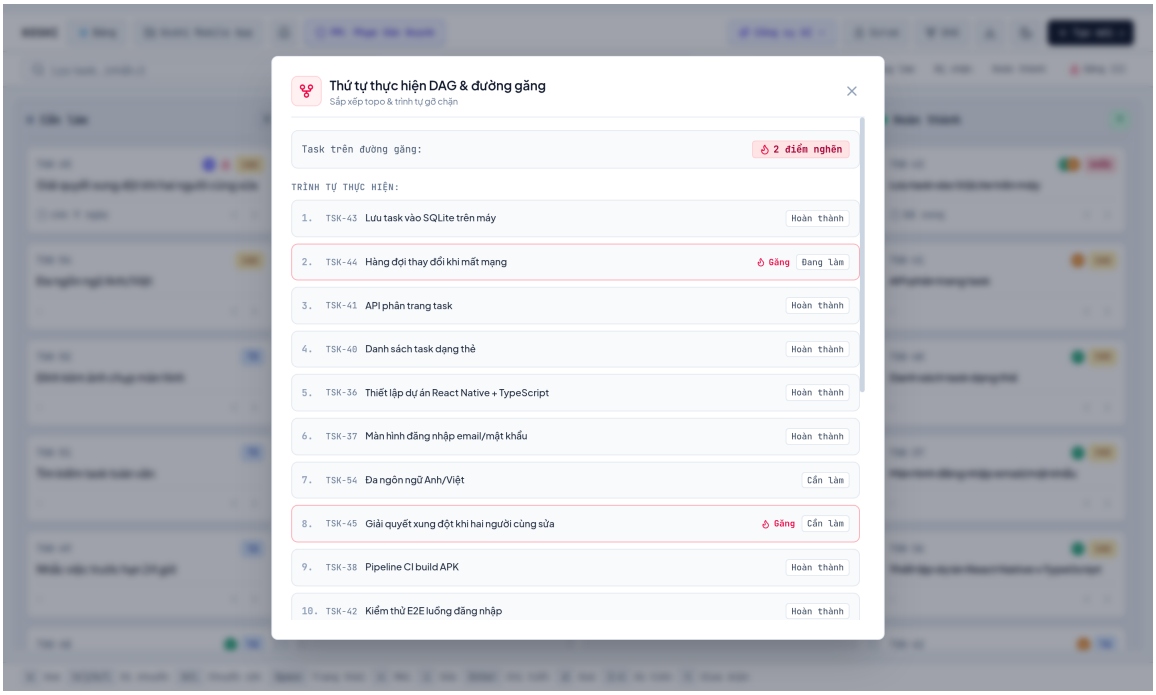
Hình 4.15. *AI phân tích khối lượng và gợi ý giao việc*



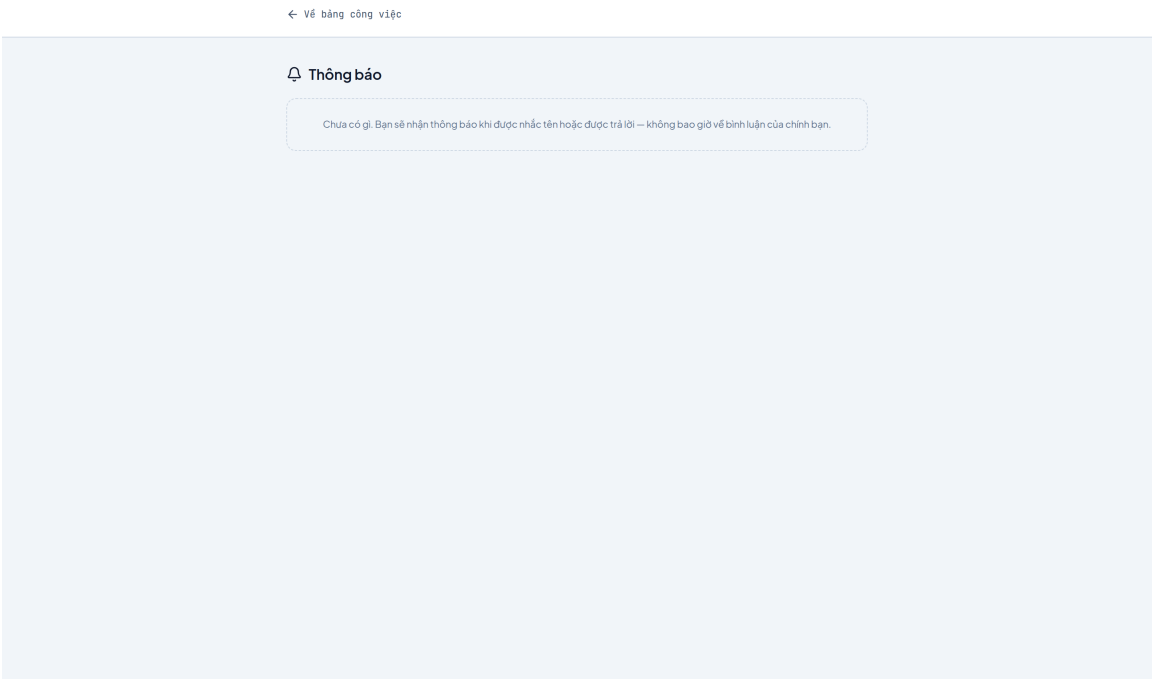
Hình 4.16. *AI chia nhỏ mục tiêu*

4.2.7. Đồ thị phụ thuộc, thông báo và hồ sơ

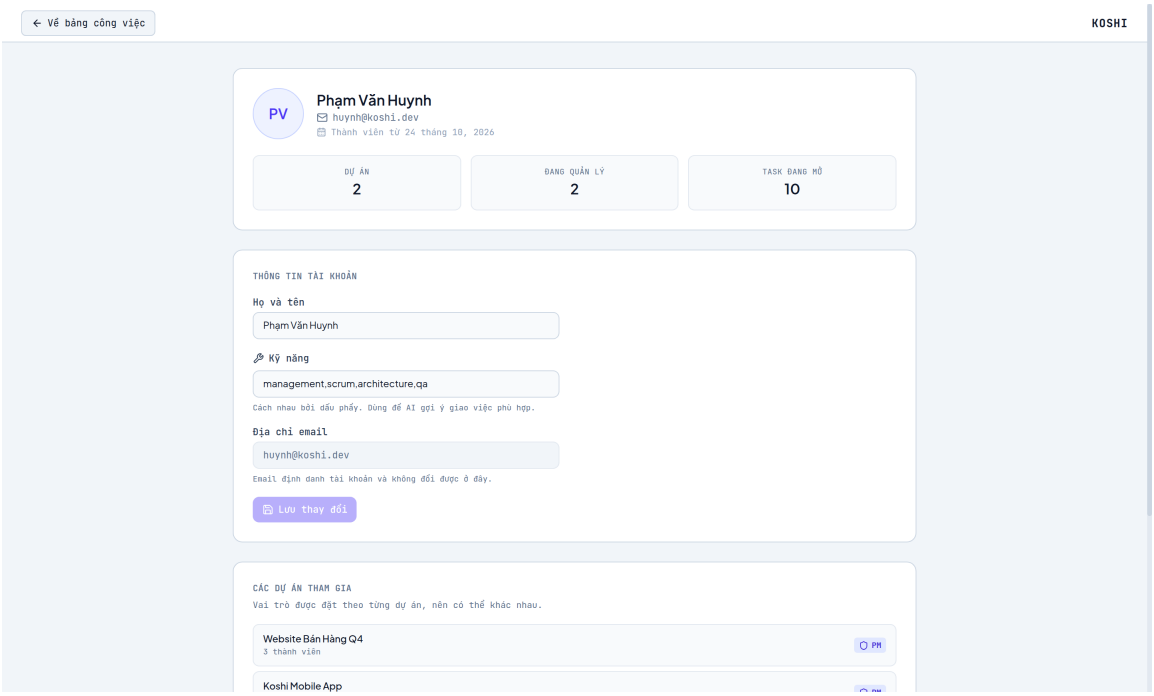
Màn hình DAG liệt kê task theo thứ tự topo và đánh dấu đường găng (Hình 4.17). Luồng thông báo báo các lần được nhắc tên, được trả lời, được giao việc và được mời—không bao giờ báo về hành động của chính người dùng (Hình 4.18). Trang hồ sơ cho sửa tên, kỹ năng (được dùng để gợi ý giao việc) và ảnh đại diện (Hình 4.19).



Hình 4.17. *Thứ tự thực hiện DAG và đường găng*



Hình 4.18. *Luồng thông báo*



Hình 4.19. *Trang hồ sơ*

4.3. Kiểm thử

4.3.1. Chiến lược

Kiểm thử gồm bốn lớp (Chương 5 phân loại theo mức đơn vị và tích hợp):

1. **Test backend** (pytest, 13 file) gọi API qua test client: mọi quy tắc vai trò và quyền (kể cả trường hợp phủ định—thành viên nhận 403, người ngoài nhận 404), vòng

trạng thái, quy tắc Scrum, lịch sử, migration, kiểm tra an toàn khi khởi động, và chuỗi AI với từng tầng được giả lập ở tầng HTTP.

2. **Test frontend** (Vitest, 19 file) bao phủ các thuật toán thuần, cả hai store, bộ xử lý phím tắt và các component trong DOM giả lập. Một test đọc template của mọi component và báo lỗi nếu còn chữ chưa được dịch.
3. **Smoke test đầu-cuối**: một script nạp dữ liệu vào instance đang chạy qua API công khai và thực hiện 32 bước kiểm tra trên mọi chức năng, kể cả bốn endpoint AI với mô hình thật.
4. **Tự động hoá trình duyệt**: Playwright điều khiển ứng dụng thật để quay video demo và chụp ảnh trong chương này, đồng thời quét từng màn hình để tìm chữ sai ngôn ngữ.

Bản thân các test cũng được kiểm tra bằng kiểm thử đột biến: cố ý xoá một điều kiện kiểm tra hoặc đổi một giá trị, và chỉ chấp nhận khi có test báo lỗi. Ví dụ, xoá điều kiện "sprint phải thuộc cùng dự án với task" làm hai test Scrum báo lỗi; xoá bộ lọc chỉ lấy thành viên đã chấp nhận làm test gợi ý giao việc báo lỗi.

4.3.2. Kết quả

Bảng 4.2. *Kết quả kiểm thử*

Bộ kiểm thử	**Số test**	**Kết quả**
Backend (pytest, SQLite)	185	đạt toàn bộ
Rào chắn agent (pytest)	56	đạt toàn bộ
Frontend (Vitest)	442	đạt toàn bộ
Kiểm tra kiểu TypeScript và build production	–	đạt
Smoke test đầu-cuối trên bản công khai	32 bước	0 lỗi
Quét lẫn ngôn ngữ trên 20 màn hình (mỗi ngôn ngữ)	20	không phát hiện

4.3.3. Các lỗi đã phát hiện

Kiểm thử đã phát hiện nhiều lỗi thật, mỗi lỗi được ghi trong sổ tay phát triển kèm nguyên nhân. Một số lỗi tiêu biểu:

Bảng 4.3. *Một số lỗi đã phát hiện và sửa*

Mã	**Lỗi**	**Cách phát hiện**
F-32	Image backend phát hành chứa file `.env` của lập trình viên cùng khoá ký phiên và một CSDL.	Kiểm tra image đã build
F-37	Lệnh triển khai trong tài liệu chép cấu hình và CSDL phát triển đè lên production.	Đọc kỹ lệnh trước khi chạy

Mã	Lỗi	Cách phát hiện
F-45	Mọi file đính kèm và ảnh đại diện hiển thị lỗi: thẻ `<img>` không gửi được header xác thực.	Kiểm thử thủ công
F-47	Khoá ngoại SQLite chưa bao giờ được bật, xoá task để lại dữ liệu mồ côi.	Khi chuyển sang MySQL
F-48	Có thể xếp task vào sprint của dự án khác.	Khi viết test Scrum
F-49	Task đã xong vẫn bị ghi "trễ 10 ngày".	Xem lại ảnh chụp
F-50	Công cụ "AI" chia nhỏ mục tiêu chưa từng gọi mô hình, chỉ trả về mẫu cố định.	Thử với mô hình thật
F-51	Gợi ý giao việc có thể chọn người chưa chấp nhận lời mời.	Rà soát mã khi kiểm thử AI
F-53	Một thay đổi được đề xuất giả lập vai trò PM ngay trên trình duyệt sau khi server từ chối.	Rà soát thay đổi được gửi tới
–	21/26 component có chữ chưa dịch, khiến giao diện lẫn hai ngôn ngữ.	Thành viên phản hồi; đã sửa và có test chặn

4.4. Triển khai

Cấu hình production chạy bốn container: MySQL, backend FastAPI, nginx cùng frontend đã build, và `cloudflared`. Script triển khai build image, xác minh trong image không có khoá bí mật hay CSDL, sao lưu CSDL, chạy migration ở một bước riêng, khởi động dịch vụ và kiểm tra instance hoạt động tốt, ở phiên bản lược đồ mới nhất và đúng tư thế production (không có tài khoản demo, không có CORS wildcard, khoá bí mật không phải mặc định). Tunnel công khai instance tại `https://koshi.huynhca.com`, nơi nhóm đã kiểm thử với ba tài khoản được tạo sẵn. Tích hợp liên tục trên GitHub Actions chạy các bộ test frontend, backend và rào chắn mỗi lần push lên nhánh chính và nhánh phát triển. Vận hành và theo dõi được trình bày ở Mục 5.6.

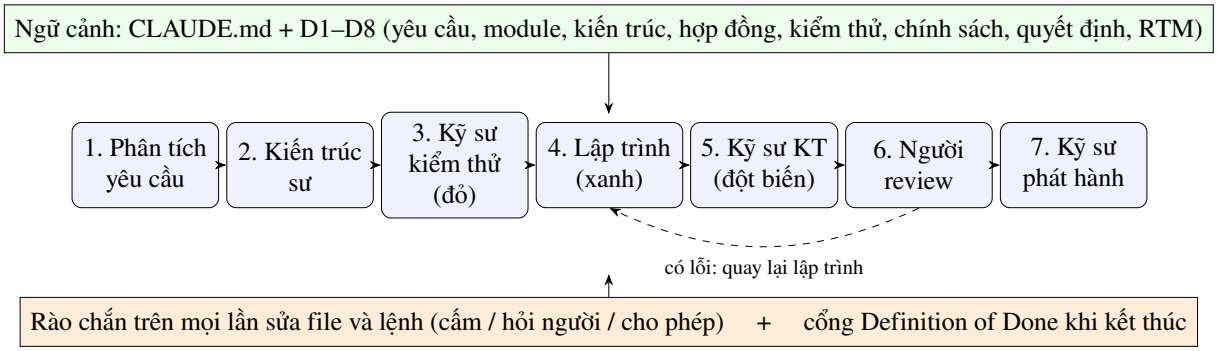
CHƯƠNG 5: AGENT AI CHO QUY TRÌNH PHÁT TRIỂN PHẦN MỀM

Hệ thống được xây dựng với sự hỗ trợ của các agent lập trình AI. Chương này mô tả cách tổ chức các agent để chúng làm việc như một nhà máy phần mềm nhỏ thay vì một trợ lý tự do: ngữ cảnh được cung cấp, các trạm của dây chuyền, các rào chắn buộc agent tuân thủ quy tắc của dự án, các mức kiểm thử phải vượt qua, và cách sản phẩm được đóng gói, vận hành và theo dõi.

5.1. Từ trợ lý đến nhà máy

Một agent lập trình AI tự làm việc có hai điểm yếu. Nó không biết dự án nên phải đoán; và nó không biết giới hạn của mình nên có thể "sửa" một quy tắc bảo mật dễ dàng như sửa lỗi chính tả. Một số lỗi trong Bảng 4.3 cho thấy cả hai: lệnh triển khai suýt đưa cấu hình phát triển lên production (F-37), và thay đổi đề xuất giả lập vai trò PM ngay trên trình duyệt (F-53). Lời giải gồm bốn phần, mỗi phần là một mục dưới đây:

1. **Kỹ thuật ngữ cảnh (context engineering)**—agent đọc đúng tài liệu trước khi hành động.
2. **Nhà máy (factory)**—công việc đi qua các trạm riêng biệt với quyền hạn riêng.
3. **Rào chắn (guardrails)**—quy tắc được thực thi bằng mã, không chỉ viết thành lời khuyên.
4. **Kiểm chứng**—kiểm thử đơn vị, tích hợp, đầu-cuối, và kiểm thử chính các bài test.



Hình 5.1. *Nhà máy agent AI: ngữ cảnh đầu vào, các trạm nối tiếp, rào chắn quanh mọi bước*

5.2. Kỹ thuật ngữ cảnh

Kho mã có một lớp tài liệu điều hướng viết cho agent trước, cho người sau. Khi bắt đầu bất kỳ việc gì, agent tự động nạp `CLAUDE.md`; tệp này liệt kê các lệnh kiểm chứng, các bất biến không được phá vỡ, và thứ tự đọc tám tài liệu:

Bảng 5.1. *Các tài liệu agent đọc và câu hỏi mỗi tài liệu trả lời*

TL	**Nội dung**	**Câu hỏi được trả lời cho agent**
D1	Yêu cầu	Hệ thống phải làm gì? (mã FR, NFR)
D2	Bản đồ module	File nào cài đặt việc này, cái gì phụ thuộc vào nó?
D3	Kiến trúc	Thay đổi của tôi phải giữ quy tắc thứ tự hay phân lớp nào?
D4	Hợp đồng API và dữ liệu	Đây có phải thay đổi hợp đồng? (nếu có: dừng và hỏi)
D5	Kiểm thử và nghiệm thu	Điều gì chứng minh nó chạy đúng; thế nào là "xong"?
D6	Rủi ro, phân quyền, chính sách	Tôi được tự làm, làm rồi báo cáo, hay phải hỏi?
D7	Sổ phát triển	Việc này đã được quyết định chưa, vì sao? Lỗi này từng gặp chưa?
D8	Ma trận truy vết	File tôi đang sửa phục vụ yêu cầu và test nào?

Hai nguyên tắc giữ cho ngữ cảnh đáng tin. Thứ nhất, mọi tài liệu được viết *dựa trên mã thật* và cập nhật trong cùng commit với mã nó mô tả; bộ tài liệu cũ từng mô tả những hành vi mà mã không có, và một agent tin vào tài liệu lỗi thời còn tệ hơn agent không có tài liệu. Thứ hai, mỗi trạm chỉ nạp những gì cần (người phân tích đọc D1, D6, D8; kiến trúc sư đọc D2, D3, D4, D6, D7), giữ ngữ cảnh gọn và tập trung.

5.3. Nhà máy: các trạm và trách nhiệm

Mỗi trạm là một định nghĩa agent riêng trong `.claude/agents/`, có chỉ dẫn, quyền dùng công cụ và mô hình riêng. Lệnh `/feature <mô tả>` đưa một yêu cầu đi qua mọi trạm theo thứ tự; `/verify` chạy mọi cổng chất lượng; `/release-check` đóng gói và chạy thử sản phẩm.

Bảng 5.2. *Các trạm của nhà máy*

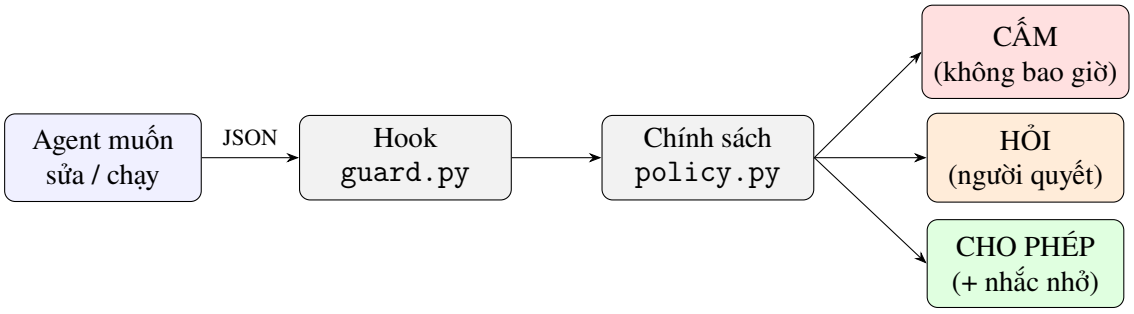
Trạm	**Sản phẩm đầu ra**	**Không được phép**
Phân tích yêu cầu	Yêu cầu có mã trong D1, tiêu chí nghiệm thu Given/When/Then trong D5, dòng mới trong D8.	Sửa mã nguồn.
Kiến trúc sư	Kế hoạch tác động: file kèm vùng tự chủ, ảnh hưởng hợp đồng, bất biến có rủi ro, danh sách test; một bản ghi quyết định trong D7.	Sửa mã nguồn; tiếp tục khi gặp mục vùng đỏ.

Trạm	Sản phẩm đầu ra	Không được phép
Kỹ sư kiểm thử	Viết test đơn vị và tích hợp *trước* (phải thất bại); sau khi cài đặt, gieo một lỗi cho mỗi test để chứng minh test bắt được.	Nới lỏng hoặc xoá assertion cũ.
Lập trình	Thay đổi nhỏ nhất để test qua, rồi build production.	Sửa file ngoài kế hoạch.
Người review	PASS, hoặc danh sách lỗi kèm kịch bản: phân quyền, hợp đồng, bí mật, injection, chính sách ngoại tuyến, dịch thuật, test, tài liệu.	Sửa bất cứ thứ gì (chỉ đọc).
Kỹ sư phát hành	Bảng cổng: bộ test đơn vị và tích hợp, test rào chắn, build, stack cấu hình production, smoke test, health và metrics.	Triển khai hoặc push khi chưa có người duyệt.

Sự tách biệt chính là cơ chế kiểm soát: người review đã tự viết mã thì không còn là người review, và người phân tích không sửa được mã thì không thể "giải quyết" yêu cầu bằng cách lặng lẽ đổi hành vi.

5.4. Rào chắn và bảo mật

Chỉ dẫn có thể bị bỏ qua; mã thì không. Vì vậy các vùng tự chủ trong D6 còn được cài đặt thành *hook* mà môi trường chạy agent gọi trước mọi lần sửa file, mọi lệnh shell, và khi agent định kết thúc (Hình 5.2). Chính sách là tập hàm thuần (`.claude/hooks/policy.py`); script hook chỉ chuyển đổi giữa giao thức JSON của môi trường và chính sách. Nếu chính script lỗi, nó từ chối—rào chắn hỏng không được biến thành cửa mở.



Hình 5.2. *Mọi lần sửa file và mọi lệnh đều qua chính sách trước khi thực hiện*

Bảng 5.3. *Một số quy tắc rào chắn*

Quyết định	**Quy tắc**	**Lý do**
Cấm	Mọi nội dung sửa có chứa API key, khoá riêng tư hoặc khoá ký JWT từng bị lộ.	Bí mật không bao giờ vào kho mã.
Cấm	Đọc hoặc in `.env`; sửa `.env`, thư mục nộp bài đã đóng băng hoặc thư mục sinh tự động.	Bí mật; sản phẩm bất biến.
Cấm	Sửa một migration đã commit.	Migration đã áp dụng là bất biến; phải thêm migration mới.
Cấm	`git push -force`; xoá đệ quy `/` hoặc `~`; lệnh triển khai tar-qua-ssh cũ.	Không thể khôi phục; F-37.
Hỏi	Sửa xác thực, mô hình dữ liệu, schema API, danh sách định dạng upload, cấu hình, CI hoặc chính rào chắn.	Vùng đỏ trong D6: bảo mật và hợp đồng.
Hỏi	`git push`, script triển khai, hạ cấp CSDL, xoá volume Docker.	Ảnh hưởng người khác hoặc xoá dữ liệu.
Cho phép + nhắc	Store, router, bản dịch, prompt AI, trang giới thiệu.	Vùng vàng: báo cáo và cập nhật tài liệu.
Chặn kết thúc	`source/` thay đổi mà không có tài liệu nào thay đổi, hoặc mã backend/frontend đổi mà không có test, hoặc model đổi mà không có migration.	Definition of Done (D5 §4).

Lần chặn đầu tiên là chính tác giả. Vài phút sau khi bật hook, rào chắn đã từ chối một lệnh do chính agent đang viết chương này gửi: một script *ghi tài liệu* có nhắc tới lệnh triển khai tar-qua-ssh cũ bị nhầm là đang chạy lệnh đó. Quy tắc được sửa để nội dung đang ghi vào file ("heredoc") được coi là dữ liệu, có thêm test cho đúng trường hợp này, và một test nữa kiểm tra lệnh nguy hiểm đặt *sau* đoạn nội dung đó vẫn bị chặn.

Bảo mật trong chính sản phẩm. Rào chắn bảo vệ kho mã; ứng dụng có lớp phòng thủ riêng, mô tả ở Chương 3 và kiểm thử ở Chương 4: phân quyền phía server cho mọi route dự án, trả 404 thay vì 403 cho người ngoài, kiểm tra khi khởi động từ chối khoá yếu, CORS wildcard hay tài khoản demo trên production, upload giới hạn theo danh sách cho phép với tên file do server sinh, và bình luận được hiển thị dạng văn bản, không bao giờ dạng HTML.

5.5. Kiểm thử đơn vị và kiểm thử tích hợp

Bảng 5.4. *Các mức kiểm thử*

Mức	**Nội dung kiểm thử**	**Công cụ**	**Số lượng**
Đơn vị (frontend)	Module thuần—sắp xếp phụ thuộc, đường găng, thứ tự khẩn cấp, bộ điều phối bàn phím, bộ phân tích mention—và hai store riêng lẻ.	Vitest	442
Đơn vị (rào chắn)	Mọi quy tắc chính sách, giao thức JSON của hook, hành vi từ chối khi lỗi.	pytest	56
Tích hợp (backend)	Router, phiên CSDL và xác thực thật qua HTTP test client; từng tầng AI giả lập ở biên HTTP; migration từ CSDL rỗng.	pytest	185
Tích hợp (giao diện)	Component gắn vào DOM giả lập cùng store.	Vitest	(trong 442)
Đầu-cuối	Instance công khai đã seed, mọi chức năng qua API, AI với mô hình thật.	script	32 bước

Chính các bài test cũng được kiểm thử. Với rào chắn, bỏ quy tắc chặn force-push và quy tắc migration làm năm test thất bại; khôi phục lại thì tất cả đạt. Tích hợp liên tục chạy ba job mỗi lần push—frontend, backend và rào chắn—nên thay đổi làm hỏng bất kỳ phần nào cũng không thể lọt qua.

5.6. Đóng gói, vận hành và theo dõi sản phẩm

5.6.1. Đóng gói

Sản phẩm gồm bốn container mô tả bằng Docker Compose: MySQL, backend FastAPI, nginx phục vụ frontend đã build, và `cloudflared` cho tunnel công khai. Image không chứa bí mật nào: `.dockerignore` loại `.env` và CSDL, và script triển khai kiểm tra lại image đã build. Bí mật được sinh riêng cho từng máy bằng `scripts/dev-env.sh`.

5.6.2. Vận hành

`scripts/local-prod.sh up` chạy migration ở một bước riêng, khởi động stack với cơ chế kiểm tra an toàn production được bật, và xác minh tư thế (khoá không mặc định, không tài khoản demo, CORS cố định, lược đồ ở phiên bản mới nhất). Docker tự khởi động lại container bị dừng, và file log được giới hạn 5×10 MB mỗi container.

5.6.3. Theo dõi

Giai đoạn này bổ sung ba tín hiệu:

- **Health có kiểm tra CSDL.** Trước đây `/api/health` vẫn báo "healthy" kể cả khi CSDL ngừng. Giờ nó chạy một truy vấn và trả HTTP 503 khi không kết nối được CSDL; healthcheck của Docker dùng endpoint này nên `docker ps` hiển thị đúng

trạng thái.

- **Mã yêu cầu và log truy cập.** Mỗi phản hồi có `X-Request-ID`; mỗi yêu cầu ghi một dòng log gồm phương thức, route, mã trạng thái và thời gian. Log ghi *mẫu* route (`/api/tasks/{task_id}`) nên không id, token hay query string nào lọt vào log.
- **Metrics.** `/api/metrics` công bố theo định dạng Prometheus: số yêu cầu theo route và trạng thái, histogram độ trễ và **số câu trả lời AI theo tầng dự phòng**. Chỉ số cuối quan trọng nhất: khi nhà cung cấp AI lỗi, mọi endpoint vẫn trả lời (theo thiết kế), nên chỉ bộ đếm này cho biết người dùng đang nhận câu trả lời dự phòng thay vì mô hình. Trên production, metrics bị ẩn (404) trừ khi gửi kèm token bí mật.

`scripts/monitor.sh` gộp các tín hiệu thành một bản tóm tắt và cảnh báo khi có lỗi server hoặc khi hơn một nửa câu trả lời AI đến từ tầng dự phòng. Kết quả trên instance công khai sau khi chạy kiểm thử đầu-cuối:

```
health      HTTP 200  (0.0026s)
container   koshi-backend-prod-local   Up (healthy)
container   koshi-db-prod-local        Up 2 days (healthy)
requests    48 total, 0 server errors
AI answers  5, 0 from the offline fallback (0%)
```

Tám test tích hợp bao phủ phần này, gồm: metrics bị ẩn khi thiếu token, mã yêu cầu độc hại bị thay thế, và id của task không bao giờ xuất hiện trong nhãn metric.

5.7. Đánh giá

Nhà máy không làm agent trở nên không bao giờ sai. Cổng kết thúc chỉ kiểm tra tài liệu và test *có thay đổi*, không kiểm tra chúng đúng; việc đánh giá đó thuộc về trạm review và cuối cùng là con người. Hook chỉ ràng buộc agent lập trình, vì vậy tích hợp liên tục chạy lại mọi bộ test một cách độc lập. Trong giới hạn đó, những quy tắc trước đây phụ thuộc vào việc agent có chịu đọc tài liệu chính sách hay không giờ được thực thi trên mọi hành động, và mỗi quy tắc đều có test đã được chứng minh là thất bại khi quy tắc bị gỡ bỏ.

KẾT LUẬN

Kết quả đạt được

Dự án đã hoàn thành một hệ thống quản lý dự án chạy được và đáp ứng các mục tiêu đề ra:

- Tài khoản không có vai trò toàn cục; dự án có vai trò PM và Thành viên theo từng dự án, lời mời phải được chấp nhận, và server kiểm tra quyền ở mọi route.
- Bảng Kanban và dạng bảng ưu tiên bàn phím, sắp xếp theo hạn chót, nhiều người thực hiện mỗi task, phụ thuộc, đường găng và màn hình DAG.
- Quy trình Scrum: vòng đời sprint, lập kế hoạch backlog, burndown, velocity và retrospective do AI viết; cùng lịch sử thay đổi cho mọi task.
- Năm chức năng AI dùng Groq với dự phòng ba tầng, đầu ra có cấu trúc được kiểm tra, và trả lời theo ngôn ngữ giao diện.
- Hoạt động local-first với chính sách ngoại tuyến an toàn; giao diện Anh/Việt hoàn chỉnh.
- Bản triển khai production tại `https://koshi.huynhca.com`, 185 test backend và 442 test frontend, tích hợp liên tục, và tám tài liệu sống truy vết mọi yêu cầu tới mã và test.
- Một "nhà máy phần mềm" agent AI: sáu trạm agent chuyên trách, rào chắn thực thi chính sách dự án trên mọi lần sửa và mọi lệnh (56 test), cùng health check, mã yêu cầu và metrics để vận hành sản phẩm.

Ngoài các chức năng, bài học giá trị nhất là về kiểm chứng. Nhiều lỗi nghiêm trọng nhất—khoá bí mật nằm trong image container, lệnh triển khai suýt ghi đè production bằng cấu hình phát triển, một chức năng "AI" chưa từng gọi mô hình, nút chuyển ngôn ngữ để lại nửa giao diện chưa dịch—đều vô hình trước một bộ test toàn xanh. Chúng chỉ được tìm thấy khi kiểm tra sản phẩm thật: soi image đã build, đọc lệnh trước khi chạy, gọi mô hình thật, quét từng màn hình. Thói quen kiểm thử đột biến cho chính các test, và ghi lại mọi quyết định cùng lý do, bắt nguồn từ những trải nghiệm đó.

Hạn chế

- Dự án chung chỉ đọc khi ngoại tuyến; chưa có thuật toán gộp các chỉnh sửa đồng thời ngoại tuyến.
- Burndown được tính theo danh sách task hiện tại của sprint, nên việc bị chuyển ra khỏi sprint sẽ biến mất khỏi lịch sử của sprint đó.
- Câu trả lời dự phòng tất định của AI hiện chỉ có tiếng Việt.
- Bản công khai chưa giới hạn số lần đăng nhập sai, và các chỉ tiêu hiệu năng trong yêu cầu (hiển thị trong một khung hình, dung lượng bộ nhớ) chưa được đo.
- Bản công khai chạy trên một máy duy nhất và không truy cập được khi máy đó tắt.

Hướng phát triển

- Dùng CRDT (kiểu dữ liệu nhân bản không xung đột) để dự án chung cũng sửa được khi ngoại tuyến.
- Nhật ký sự kiện theo sprint để có lịch sử burndown chính xác, và màn hình daily scrum.
- Giới hạn tần suất đăng nhập, khoá tài khoản; chuyển sang hosting có sao lưu tự động.
- Ứng dụng di động gốc và tích hợp với nền tảng Git (tự động liên kết commit, pull request với task).

TÀI LIỆU THAM KHẢO

[1] M. Kleppmann, A. Wiggins, P. van Hardenberg, and M. McGranaghan, "Local-first software: You own your data, in spite of the cloud," in *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward!)*, 2019, pp. 154–178. DOI: 10.1145/3359591.3359737

[2] R. A. DeMillo, R. J. Lipton, and F. G. Sayward, "Hints on test data selection: Help for the practicing programmer," *Computer*, vol. 11, no. 4, pp. 34–41, 1978. DOI: 10.1109/C-M.1978.218136

[3] D. J. Anderson, *Kanban: Successful Evolutionary Change for Your Technology Business*. Blue Hole Press, 2010.

[4] K. Schwaber and J. Sutherland, *The scrum guide: The definitive guide to scrum: The rules of the game*, https://scrumguides.org/scrum-guide.html, 2020.

[5] A. B. Kahn, "Topological sorting of large networks," *Communications of the ACM*, vol. 5, no. 11, pp. 558–562, 1962. DOI: 10.1145/368996.369025

[6] J. E. Kelley and M. R. Walker, "Critical-path planning and scheduling," in *Proceedings of the Eastern Joint IRE-AIEE-ACM Computer Conference*, 1959, pp. 160–173. DOI: 10.1145/1460299.1460318

[7] A. Vaswani et al., "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[8] OpenAI, *Gpt-oss-120b & gpt-oss-20b model card*, https://openai.com/index/gpt-oss-model-card/, 2025.

[9] Groq, Inc., *Groqcloud api documentation: Openai compatibility*, https://console.groq.com/docs/openai, Accessed October 2026.

[10] Vue.js Team, *Vue.js documentation*, https://vuejs.org/guide/, Accessed October 2026.

[11] S. Ramírez, *Fastapi documentation*, https://fastapi.tiangolo.com/, Accessed October 2026.

[12] M. B. Jones, J. Bradley, and N. Sakimura, *JSON Web Token (JWT)*, 2015. DOI: 10.17487/RFC7519

[13] Cloudflare, Inc., *Cloudflare tunnel documentation*, https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/, Accessed October 2026.

[14] OWASP Foundation, *Owasp top 10:2021*, https://owasp.org/Top10/, 2021.

[15] I. Sommerville, *Software Engineering*, 10th ed. Pearson, 2016.

[16] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 9th ed. McGraw-Hill Education, 2020.

[17] K. Beck, *Test-Driven Development: By Example*. Addison-Wesley, 2002.

[18] ISO/IEC/IEEE, *Iso/iec/ieee 29148:2018 systems and software engineering – life cycle processes – requirements engineering*, 2018.

NHẬN XÉT CỦA GIẢNG VIÊN

. . . . . .

. . . . . .

. . . . . .

. . . . . .

. . . . . .

. . . . . .

. . . . . .

. . . . . .

. . . . . .

. . . . . .

. . . . . .

. . . . . .

Thái Nguyên, tháng … năm …

Giáo viên hướng dẫn